

비개발자를 위한 바이트코딩 입문

with 클로드 코드

AI와 대화하며 실제로 쓸 수 있는 서비스를 만드는 3일 집중 과정 — 이론부터 배포·운영까지

과정 소개

WITH CLAUDE CODE · 3-DAY INTENSIVE

AI와 대화하며 실제로 쓸 수 있는 서비스를 만드는 3일. 코딩 경험이 없어도 괜찮습니다 — 이론은 배운 즉시 손으로 만들어 확인합니다.

이 과정의 3가지 약속

- ① 이론 → 즉시 실습 — 모든 이론은 40분 이내로 끊고, 바로 써먹는 실습이 따라온다
- ② 웹페이지에서 끝나지 않는다 — 크롬 익스텐션, 데이터가 저장되는 앱까지 "실제로 쓰는 형태"로
- ③ 진짜 저장되는 앱 — 3일차에 DB를 붙인 노션 클론을 만들어 인터넷에 배포한다

⚠️ 수업 전일까지 끝내야 할 준비물(계정 가입 등)이 있습니다 — 부록 「수강 전 준비 체크리스트」
먼저 확인

🎯 실습 결과물은 패들렛에 올려 함께 공유해요

3일의 여정

(도식: 3일 로드맵 도식)

- DAY 1 · 기본기 — 개발환경 · 웹의 구조 · 프롬프트 → 프로필 페이지 공개 + 게임 만들며 디버깅
- DAY 2 · 연결 — API → 크롬 익스텐션, 공공 데이터 시각화+AI, 테스트 결제까지
- DAY 3 · 저장과 공개 — DB 입문 → 진짜 저장되는 노션 클론 + Vercel 배포

이 교안 사용법

- 왼쪽 목차에서 진행 중인 세션 클릭 → 이동, 상단 검색창에 "배포·API·결제" 등으로 필터
- 어두운 상자 안 예시 프롬프트는 그대로 복사해 Claude에게 붙여넣어도 됨
- 오른쪽 메모장은 자동 저장 — 끝나면 MD로 다운로드

💡 목차 맨 위 「이론 기초」 3장은 시간표에 묶인 세션이 아니라 3일 내내 참조하는 배경 지식입니다.

이론 기초

3일 내내 참조하는 배경 지식

AI가 답하는 원리

확률 기계에서 프롬프트까지

이론 기초 01

AI가 왜 그렇게 답하는지 알면, 어떻게 시켜야 하는지가 저절로 따라옵니다.

LLM은 "다음 말"을 확률로 고르는 기계

- 110년 동안의 질문은 하나였습니다 — "다음에 올 말은 무엇인가?"
- 마르코프(1913) → 샤페론(1948) → 신경망 언어모델(2003) → 트랜스포머(2017) → ChatGPT(2022) → 에이전트(지금)
- 기술은 바뀌어도 원리는 그대로 — 앞의 말을 보고 다음 말을 확률로 예측한다

(도식: 문맥이 쌓일수록 확률 분포가 좁아지는 과정)

왜 같은 질문에 매번 다른 답이 — 샘플링

- AI는 1등만 고르지 않고 분포에서 추출(샘플링)한다 — 가끔 2등·3등도 뽑힌다
- 답이 매번 달라지는 건 고장이 아니라 설계다
- 창의성과 환각은 같은 다이얼의 양 끝 — 추출 폭을 넓히면 창의적, 너무 넓히면 헛소리

환각(할루시네이션) — 원리상 피할 수 없는 것

- 학습에 없는 걸 물어도 AI는 "그럴듯한 다음 단어"를 지어낸다 — 없는 함수·링크·판례를 자신 있게
- 확률 예측 방식의 필연적 결과라서, 프롬프트를 잘 쓴다고 사라지지 않는다
- ⚠ 이 문제의 답은 과정 마지막에 — 도구로 사실을 확인하게 만드는 것(이론 03 RAG). 지금은 "이런 문제가 있다"만 기억

프롬프트는 "확률을 좁히는 행위"

	막연한 요청	구체적인 요청
예시	"맛있는 거 주세요"	"매운 것 빼고, 둘이서 3만 원어치"
확률 분포	평평함 — 후보가 너무 많다	뾰족함 — 후보가 좁혀진다
결과	그럴듯한 평균이 나온다	내가 원하던 것이 나온다

PTCF — 프롬프트를 구체적으로 쓰는 틀

요소	묻는 질문	예시
Persona	누구로서?	"당신은 동네 카페 마케터"
Task	무엇을?	"홍보 문구를 3개 써 주세요"
Context	어떤 상황에서?	"손님 대부분 출근길 30대 직장인"
Format	어떤 모양으로?	"문구당 두 문장, 해시태그 3개씩"

- 매번 다 채울 필요는 없다 — 쓰기 전에 네 칸을 떠올리는 습관이 핵심

TCREI — 프롬프트를 계속 고쳐가는 틀

요소	뜻	성격
Task	할 일을 한 문장으로	넣는 재료
Context	대상·배경·조건	넣는 재료
References	예시·원문·양식을 함께	넣는 재료
Evaluate	결과를 채점한다	받은 뒤 사람의 행동
Iterate	어긋난 부분만 다시 시킨다	받은 뒤 사람의 행동

- 잘 쓰는 사람과 못 쓰는 사람은 위의 두 칸(E·I)에서 갈린다

EXAMPLE — Evaluate → Iterate

1차 결과 채점: 순서 ○, 색 ○, 대표 메뉴 × (세상 카페의 평균이 나옴)

→ 어긋난 칸만 짚어서 다시:

"대표 메뉴를 흑임자 라떼·수제 스콘·핸드드립으로 바꿔 주세요. 나머지는 그대로."

프롬프트를 구조화하는 3가지 형식

형식 없는 긴 글은 AI에게 "다 한 줄"로 보입니다. 구역을 나눠주면 조건이 섞이지 않습니다.

형식	비유	언제	대표 예
마크다운	형광펜	매일 씀	CLAUDE.md, 이 교안
XML 태그	벽	가끔	<자료>붙여넣은 글</자료>
JSON	자물쇠	알아만 두기	API 응답, 툴 정의서

💡 긴 자료는 <자료> ... </자료> 로 감싸세요 — AI가 자료 속 문장을 지시로 오해하는 사고를 막아줍니다.

마크다운 최소 문법 5개 — 이거면 충분

쓰는 법	결과
# 제목	제목
굵게	강조
- 항목	목록
```코드```	코드 블록
[이름](주소)	링크



## 이론 01 — 더 깊이 (참고 자료)

- Attention Is All You Need — 트랜스포머 원논문 (2017), [arxiv.org](https://arxiv.org/abs/1706.03762)
- 프롬프트 엔지니어링 공식 문서 (Anthropic)
- 구글 공식 프롬프트 가이드 (PTCF 출처)
- Google Prompting Essentials (TCREI 출처)

## 에이전트와 하네스

### *이론 기초 02*

확률 기계에 도구·기억·순환을 붙이면 에이전트가 됩니다. 그 에이전트가 일할 환경을 짜는 것이 하네스 엔지니어링입니다.

# 챗봇과 에이전트 — 기준은 딱 하나

기능 개수가 아니라 ****"다음 단계를 누가 정하는가"****로 가릅니다.

	챗봇	에이전트
다음 단계 결정	사람	AI
비유	말하는 백과사전	일을 통째로 맡은 담당자
"매출 요약해줘"	방법 알려주고 끝	찾기→계산→표→검토를 스스로 반복

 **엔트로픽 정의:** "환경 피드백을 받으며 루프 안에서 도구를 쓰는 LLM"

## 에이전트의 구성요소

구성요소	역할	없으면
추론 모델	다음에 뭘 할지 판단하는 두뇌	아무 결정도 못 함
툴(도구)	세상을 읽고 바꾸는 손	판단만 하고 안 바뀜
메모리	한 일과 알게 된 것을 담는 곳	매번 처음부터 다시
(루프)	될 때까지 반복하고 다 되면 멈춤	챗봇에 그침

## "에이전틱하다"는 무슨 뜻일까

- 시킨 하나만 하고 멈추는 게 아니라, 목표를 위해 스스로 여러 단계를 밟을 때 "에이전틱하다"
- 비유 — 심부름에서 "재료 없으면 사 오고, 대체품 찾고, 다 되면 알려주는" 사람
- "재료 없던데요?" 하고 멈추면 에이전틱하지 않은 것 — 명령받으면 계획→실행→판단을 반복

## GRAM 루프 — 에이전트가 도는 순환

(도식: GRAM 루프 순환 도식)

- 클로드 코드 화면의 "파일 읽는 중", "명령 실행 중"이 이 순환 — 무서운 로그가 아니라 진행 표시
- GRAM은 외울 표가 아니라 이름표 — "지금 Reason 중이구나"라고 부를 수 있으면 충분
- 영어 자료: agent loop, 학술 원전은 ReAct(Yao et al. 2022)

## 툴로 쓸 수 있는 것들

- 파일 읽기·쓰기·수정 / 명령 실행(터미널) / 검색(파일·웹) / 브라우저 조작 / 외부 서비스(노션·깃허브·DB)
- 외부 서비스를 붙이는 표준 규격이 **MCP**(Model Context Protocol) — AI에게 손·눈을 달아주는 것

# 메모리 — 단기와 장기

	단기 메모리	장기 메모리
실체	컨텍스트 윈도우 — 지금 대화에 올라온 것	기록 — 파일에 적어둔 것
수명	대화 끝나면 사라짐	다음 대화에서도 읽어 씀
예	방금 대화, 읽은 파일	CLAUDE.md, 작업 노트, 회고



## 컨텍스트 — 한국어로 "맥락"

- AI가 지금 대화를 이해하려고 참고하는 모든 것 — 질문·앞선 대화·읽은 파일·붙여넣은 자료
- AI에겐 눈치도 배경지식도 없어 컨텍스트에 올라온 것만 안다 — "무엇을 넣어주느냐"가 결과를 좌우
- 이 컨텍스트를 담는 그릇의 크기가 컨텍스트 윈도우

💡 [캡처: 클로드 공식 컨텍스트 윈도우 설명 이미지]

## 컨텍스트 윈도우 — 왜 새 주제는 새 채팅인가

- AI가 한 번에 기억할 수 있는 양 — 무한하지 않다
- 짝 차기 **전부터** 품질이 떨어진다 — 시행착오·버린 코드가 쌓이면 답이 평균으로 흐른다
- 그래서 주제가 바뀌면 새 채팅 — 넘쳐서가 아니라 흐려지기 전에 끊는 것

## 토큰 — 세 갈래로 나눠 보기

구분	무엇인가	알아둘 점
입력	프롬프트·파일·이전 대화	가장 싸다. 톨 결과도 다음 턴엔 입력
추론	답 전에 속으로 생각한 것	출력으로 과금 — 가장 비싸다
출력	화면에 보이는 답변	입력의 약 5배

💡 반복해 주면 캐싱되어 입력 비용 1/10. "많이 읽히기"보다 "길게 생각하고 길게 답하기"가 훨씬  
비쌘

## 컨텍스트 엔지니어링

- "이 작업에 어떤 컨텍스트를 줘야 하는가?" — 필요한 건 넣고, 없는 건 빼고, 순서를 정하는 것
- 이 고민 자체를 컨텍스트 엔지니어링이라 부른다
- 많이 넣는 게 좋은 게 아니다 — 관계없는 자료는 분포를 흐리고 비용만 늘린다

# 에이전트 문서 — 규칙을 어디에 적어두나

로드되는 시점이 서로 다르다는 게 핵심입니다.

(도식: 에이전트 문서의 3단 로딩 구조)

문서	정체	로드 시점
CLAUDE.md (범용: AGENTS.md)	프로젝트 규칙 문서	새 작업마다 항상
Rules	주제별 규칙 파일	해당 경로 파일 다룰 때만
Skills ( SKILL.md )	작업 절차 지시서	필요할 때 찾아서

## 문서 vs 훅

- 지침서가 길면 실제 작업에 쓸 컨텍스트가 줄어든다 → 개념 단위로 쪼개 필요할 때 찾아 쓰게 (Skills 가 그 해법)
- ⚠ Hooks는 문서가 아니다 — 정해진 시점에 반드시 실행되는 자동 명령(코드), 컨텍스트도 안 차지
- 문서 3종이 부탁이라면 훅은 강제

## 하네스 엔지니어링

- 하네스 = 말을 부릴 때 쓰는 장비. 에이전트가 일할 프로젝트 환경 전체를 설계하는 것
- 무엇을 짜나 — 규칙 문서, 권한 범위, 도구, 강제 장치(혹), 결과 확인 방법
- 같은 AI라도 잘 짜인 하네스에서 훨씬 잘 일한다 — 모델보다 환경을 고치는 게 빠를 때가 많다

## 이론 02 — 더 깊이 (참고 자료)

- Building Effective Agents (Anthropic 공식, 영문)
- ReAct — GRAM 루프의 학술 원전 (Yao et al. 2022)
- CLAUDE.md(메모리) 공식 문서
- Skill 공식 문서 / Hooks 공식 가이드



## RAG와 에이전틱 서치

### 환각에 대한 답

#### *이론 기초 03*

이론 01의 문제 — AI는 모르는 것도 그럴듯하게 지어냅니다. 그 답이 여기 있습니다.

## 답은 "지어내지 말고, 찾아보게 하는 것"

- 확률로 만들어내는 대신, 실제 자료를 읽고 그걸 근거로 답하게 만든다
- 이게 도구(툴)가 존재하는 근본 이유 — 검색·파일 읽기·코드 실행으로 사실 확인
- 앤트로픽이 에이전트를 "환경 피드백을 받으며 루프를 도는 LLM"이라 정의하는 이유 (환경 피드백 = 사실 확인)

## RAG — 검색해서 근거를 붙인 뒤 답하기

- RAG = Retrieval-Augmented Generation, "검색으로 보강한 생성"
- 흐름: 질문 → 관련 자료 먼저 찾고 → 함께 넣어서 → 답한다
- 오해 — RAG는 벡터 DB를 뜻하는 게 아니다. 클로드 코드가 내 폴더에서 파일을 찾아 읽는 것도 넓은 RAG

⚠ RAG는 환각을 없애지 않습니다. 찾아온 자료가 틀릴 수도, 잘못 읽을 수도 있습니다. RAG는 근거를 붙여 검증 가능하게 만들 뿐 — 오히려 결과를 확인하는 습관이 더 중요해집니다.

## 에이전틱 서치 — 검색을 에이전트가 스스로

(도식: RAG와 에이전틱 서치 비교)

- RAG는 정해진 방식으로 한 번 찾아 넣는다. 에이전틱 서치는 무엇을 찾을지 스스로 판단하고 부족하면 다시 찾는다
- 대체가 아니라 겹쳐 쓰는 관계 — 에이전틱 서치 = RAG + GRAM 루프
- 기준은 낮익다 — "다음 단계를 누가 정하는가"(이론 02)

## LLM 위키 — 찾아 쓸 지식을 아예 쌓아두기

- RAG가 "필요할 때 찾아온다"면, 찾아올 대상을 미리 잘 만들어두자는 흐름 = LLM 위키
- 거창하지 않다 — 마크다운 파일을 폴더로 정리한 것 (용어·절차·과거 사고 기록)
- CLAUDE.md도 그중 하나. 구글이 공통 규격으로 정리한 것이 OKF(Open Knowledge Format)
- 회사에 진짜 필요한 건 내부 지식 — AI가 학습한 적 없다. 위키에 쌓아두면 매번 설명 안 해도 됨

💡 LLM 위키는 이론 02의 장기 메모리를 개인에서 팀·조직 단위로 넓힌 것입니다.

## 컨텍스트 엔지니어링과의 관계

- 이론 02: 컨텍스트 엔지니어링 = "어떤 컨텍스트를 줄지 고민하는 것"
  - RAG는 그 고민을 자동화한 것 — 사람이 고르던 자료를 시스템이 질문에 맞춰 대신 찾아 넣음
  - 다 넣으면 흐려지고, 안 넣으면 지어낸다 — 그 사이를 푸는 게 RAG
- 💡 3일 과정 한 줄 요약 — 확률 기계(01) → 잘 시키는 법(프롬프트) → 도구·순환을 붙여 에이전트로 (02) → 사실을 확인하게(03)

## 이론 03 — 더 깊이 (참고 자료)

- Building Effective Agents (Anthropic 공식, 영문)
- RAG 원논문 — Retrieval-Augmented Generation (Lewis et al. 2020)
- Open Knowledge Format — LLM 위키 규격화 문서 (구글 클라우드)

# DAY 1 — 기본기

*만들고, 고치고, 배포한다*



## 기본기 — 만들고, 고치고, 배포한다

### DAY 1

오늘의 목표: 아이디어 → 배포된 웹페이지까지 전체 사이클을 두 번 경험한다.

산출물 — 배포된 프로필 페이지 URL 1개 + 플레이 가능한 게임 1개

오전엔 도구·재료·기술을 익히고, 오후엔 실제 프로젝트 두 개를 완성합니다.

## DAY 1 시간표 (1/2)

시간	구분	세션
09:00-09:30	이론	오리엔테이션 · 바이브코딩이란
09:30-10:20	이론+실습	개발환경 구성 + 첫 페이지
10:30-11:10	이론+실습	웹의 구조
11:10-12:00	이론+실습	프롬프트 엔지니어링 + 토큰 절약

## DAY 1 시간표 (2/2)

시간	구분	세션
13:00-13:30	이론	바이브코딩 플로우 + 깃
13:30-15:00	프로젝트	프로젝트 1 · 프로필 페이지
15:10-16:40	프로젝트	프로젝트 2 · 고전게임 만들기
16:40-17:00	회고	Day 1 회고

## 오리엔테이션 · 바이브코딩이란

DAY 1 · 09:00–09:30 · 이론

코드를 직접 타이핑하지 않아도 됩니다. AI에게 한국어로 설명해서 만드는 개발 — 그게 바이브코딩입니다.

(도식: 바이브코딩의 순환 구조 도식)

## 바이트코딩 — 핵심 내용

- 바이트코딩 = "이런 걸 만들어줘"라고 말하면 AI가 코드를 대신 써주는 방식 — 우리는 "시키고 확인하는 사람"
- 필요한 실력 — 문법 암기 ❌ / 원하는 걸 정확히 설명하고 결과를 판단하는 능력 ○
- 코드를 몰라도 시작하지만, 웹의 큰 그림은 알아야 잘 시킨다 (오전에 배움)
- 로드맵: Day1 공개 → Day2 API 연결 → Day3 DB로 저장되는 앱

## "바이브코딩"이라는 말은 어디서

코드를 직접 써 내려가는 일이 아니라, 코딩 에이전트와 대화하며 만들고 싶은 결과물을 정확히 설명하고 이해시키는 일.

- OpenAI 창립 멤버 안드레이 카파시가 2025년 초 처음 꺼낸 말 — "흐름(vibe)에 몸을 맡기고 AI에게 시키며 만든다"
- 핵심은 타이핑이 아니라 "무엇을, 왜 만들고 싶은지"를 또렷하게 전달하는 힘 (주방장에게 정확히 주문하는 손님)
- "설명하고 → 확인하고 → 다시 요청하는" 감각이 3일 내내 기를 진짜 실력

# 일반 챗봇과 뭐가 다른가

	일반 챗봇	클로드 코드
역할	조언하는 컨설턴트	직접 일하는 개발자
코드	보여주기만 (내가 복붙)	파일을 스스로 만들고 고침
실행	불가	터미널에서 직접 실행·확인
에러	설명해줌	스크린샷 붙이면 즉시 분석·수정
체감	매번 복붙 왕복	"가계부 만들어줘" → 2~5분 완성

## 미리 알아두면 마음 편한 것 2가지

- AI는 확률 기계다 — 매번 조금씩 다른 답은 고장이 아니라 설계. 맥락을 많이 줄수록 정확해진다
- 정말 어려운 건 개발이 아니라 **가입·설치·연결**이다 — 막히는 게 정상, 여러분 잘못이 아니니 바로 손들자



## 이 교육의 "바이브코딩 6단계"

아래 6단계를 한 바퀴 돌리면 결과물 하나가 나옵니다. 3일 동안 이 바퀴를 계속 돌립니다.

(도식: 폴더 생성 → Plan → Build → 검토·개선 → 배포 → 공유, 반복)

💡 한 번 쓰고 버리는 순서가 아닙니다 — 작은 결과물마다 ①~⑥을 반복. 몸에 배면 여러분만의 서비스도 같은 방식으로.

💡 오늘의 약속 — 퇴근 전에 각자의 웹페이지가 인터넷에 올라갑니다.

## 오리엔테이션 — 더 깊이 (참고 자료)

- Claude 요금제 안내 ([claude.com](https://claude.com))
- AI 에이전트란 무엇인가 (Anthropic 공식 글, 영문)

## 개발환경 구성 + 첫 페이지 만들기

*DAY 1 · 09:30–10:20 · 이론 · 실습*

Claude를 쓰는 3가지 방법을 알아보고, 수업용 환경을 세팅해 첫 결과물을 띄웁니다.

## Claude를 쓰는 3가지 창구

환경	모습	어울리는 일
데스크톱 앱	메신저처럼 대화	질문, 아이디어, 문서
VS Code 확장	편집기 안의 Claude	파일 보면서 시키기
Claude Code (터미널)	명령어 창의 에이전트	파일 생성·수정 자동 — 가장 강력

- 수업은 한 가지 환경으로 전원 통일 — 지금부터 다 같이 설치

## 따라하기 ① — 윈도우에 Claude Code 설치

1. Claude 유료 구독(Pro 이상) 로그인 가능 확인 (무료면 지금 업그레이드)
2. 윈도우 키 → "powershell" → Enter
3. 아래 명령어 붙여넣고(우클릭=붙여넣기) Enter — 자동 설치
4. 설치 후 PowerShell 창 닫았다 새로 열기
5. `claude --version` → 버전 나오면 성공
6. `claude` → 브라우저 로그인 → 코드 붙여넣기 → "Login successful"

```
irm https://claude.ai/install.ps1 | iex
```

- 참고: Node.js 있으면 `npm install -g @anthropic-ai/claude-code` 도 가능 (오늘은 위 방식 통일)

## 환경변수 — 컴퓨터 전체가 기억하는 설정

- 환경변수 = 특정 프로그램이 아니라 **컴퓨터 전체에 붙여두는 메모**. 어떤 프로그램이든 읽을 수 있다
- 대표적인 게 **Path** — "명령어를 치면 어느 폴더에서 찾을지" 목록. `claude` 만 쳐도 실행되는 이유
- `claude` 명령을 못 찾는 에러 = Path에 설치 폴더가 등록 안 된 것 → 직접 등록하면 해결
- 내일(Day 2) API 키도 환경변수로 보관 — 코드에 키를 안 적는 안전한 방법

## 따라하기 ② — Path 등록하기

1. `claude --version` 이 잘 나왔다면 건너뛰어도 됨 (에러 날 때만)
2. 윈도우 키 → "환경 변수" 검색 → "시스템 환경 변수 편집" → [환경 변수] 버튼
3. "사용자 변수"에서 Path 선택 → [편집]
4. [새로 만들기] → 아래 경로 입력 → 확인
5. PowerShell 새로 열고 `claude --version` 확인

```
%USERPROFILE%\local\bin
```

💡 새 환경변수도 같은 창에서 — "사용자 변수"의 [새로 만들기]. 내일 API 키 보관 때 다시 만납니다.

## 터미널이란 — 마우스 대신 '말로' 시키는 창

- 터미널(PowerShell) = 마우스로 하던 일을 글자 명령어로 시키는 프로그램 (더블클릭 대신 `cd` 폴더명 )
- 검은 화면이 낯설 뿐 어렵지 않다 — 외울 명령어는 대여섯 개, 나머지는 Claude가 알아서
- 왜 배우나 — 가장 강력한 Claude Code가 이 터미널 위에서 돈다

💡 [캡처/짤: 매트릭스에서 트리니티가 터미널로 해킹하는 장면] — 영화 속 그 검은 창이 바로 터미널



## 왜 코딩 에이전트에겐 터미널이 잘 맞나

- Claude 같은 LLM은 '언어'를 학습한 모델 — 그림보다 글로 주고받을 때 가장 빠르고 정확
- 터미널은 모든 작업이 글자로 오간다 → 마우스 화면 조작보다 AI에겐 훨씬 효율적
- 정리 — 사람에게겐 마우스, AI에겐 터미널. 그래서 일 잘하는 AI 도구가 터미널에서 돈다

## 자주 쓰는 터미널 명령어 (이것만)

명령어	하는 일	예시
<code>cd</code> 폴더명	폴더 안으로 들어가기	<code>cd vibe-day1</code>
<code>cd ..</code>	위 폴더로 나가기	<code>cd ..</code>
<code>ls</code> (윈도우 <code>dir</code> )	파일 목록 보기	<code>ls</code>
<code>pwd</code>	지금 위치 전체 주소	<code>pwd</code>
<code>mkdir</code> 이름	새 폴더 만들기	<code>mkdir images</code>
<code>cls</code> / <code>clear</code>	화면 청소	<code>cls</code>

- 제일 많이 쓰는 건 `cd` 하나. 나머지는 "이런 게 있었지" 정도면 됨

# 절대경로 vs 상대경로

비유 — 집 알려주기: ① "서울시 ○○구 12"(절대) ② "오른쪽 두 번째 골목"(상대)

구분	뜻	예시
절대경로	뿌리(드라이브)부터 전체 주소 — 딱 한 곳	C:\Users\내이름\...\index.html
상대경로	지금 위치 기준 짧은 주소	index.html · images/logo.png · ../

- Claude가 "경로를 못 찾겠다"면 대부분 위치 착각 → `pwd` 확인 또는 절대경로로

## 로컬호스트 · 포트 · 서버

- 서버 = 24시간 켜져 있는 컴퓨터. 인스타 접속 = 저쪽 서버가 화면을 내려주는 것
- 로컬호스트(localhost) = 내 컴퓨터 자신(127.0.0.1). 공개 전 나만 보는 '내 컴퓨터 미리보기'
- 포트(port) = 건물의 문 번호. 여러 프로그램이 "3000번 문, 5500번 문" 식으로 나눠 씀
- `http://localhost:3000` = "내 컴퓨터의 3000번 문으로 들어와" — 겁먹지 말고 주소창에 붙여넣기

💡 서버 = 늘 켜진 컴퓨터, 로컬호스트 = 내 컴퓨터, 포트 = 문 번호

## 쓰기 전에 알아둘 것

- 반드시 프로젝트 폴더 안에서 실행 — 클로드 코드는 "열려 있는 폴더" 기준으로 일함. 바탕화면에서 실행하면 바탕화면 전체가 프로젝트가 됨 (실수 1위)
- 스크린샷 한 장이 설명 열 줄보다 낫다 — `Win+Shift+S` 로 캡처, 클로드 코드 창에 `Ctrl+V`

## 허락을 물어보면 이렇게 답한다

입력	뜻	언제
y	이번 한 번 허락	처음엔 하나씩 확인하며
n	거부	의도와 다를 때
a	이 대화에선 계속 허락	믿고 맡길 때 (속도↑)

## 기본 명령어

명령어	뜻
<code>/init</code>	"프로젝트 파악해 설명서(CLAUDE.md) 만들어줘"
<code>/clear</code>	"대화 비우고 새로 시작"
<code>/compact</code>	"길어진 대화 요약해서 이어가기"
계획 모드	"실행 전에 계획부터 보여줘"

- 프롬프트 = AI에게 보내는 요청 문장. 3일 내내 주력 도구

# 슬래시(/) 명령어 — 더 알아두면 좋은 것

입력창에 / 만 쳐도 목록이 뜹니다.

명령어	하는 일
/help	쓸 수 있는 명령어·사용법
/model	AI 모델 고르기 (똑똑함↔속도)
/cost	이 대화의 사용량
/resume	이전 대화 이어서 열기
Esc 키	AI가 엉뚱할 때 즉시 멈추기



# 클로드 코드의 4가지 실행 모드

`Shift+Tab` 을 누를 때마다 모드가 차례로 바뀝니다.

모드	무엇	언제
플랜	건드리지 않고 계획만 (읽기 전용)	큰 작업 전 — 강력 추천
편집(수락)	바꾸기 전 매번 y/n	기본값, 가장 안전
오토(자동수락)	일일이 안 묻고 진행	익숙해졌을 때
바이패스	권한 확인 다 건너뛰	쓰지 않기

 바이패스( `--dangerously-skip-permissions` )는 파일 삭제까지 안 묻고 실행 — 수업에서는 켜지 마세요.

## 플랜 모드 + 되묻기(AskUserQuestion)

- 큰 걸 시킬 땐 바로 만들게 말고 플랜 모드로 계획부터 → 승인해야 시작하니 되돌리기가 확 준다
- "애매하면 되물어봐"를 붙이면 Claude가 넘겨짚는 대신 선택지를 묻음 — 긴 프롬프트 없이 방향이 좁혀짐

### EXAMPLE PROMPT (플랜 모드 + 되묻기)

동네 카페 소개 페이지를 만들 거야. 바로 만들지 말고 먼저 계획을 세워서 보여줘.  
정하기 애매한 부분(색감, 페이지 구성, 담을 내용)은 나에게 선택지로 되물어봐.  
내가 계획에 OK 하면 그때 만들기 시작해줘.

## 클로드 코드는 '지침서'부터 읽고 시작한다

- 일을 시작할 때 프로젝트 폴더의 규칙 문서( `CLAUDE.md` )를 먼저 읽고, 그 맥락 위에서 요청을 처리
- "한국어로 답해줘" 같은 반복 지시를 안 해도 되고, `/clear` 해도 규칙은 유지 (새 대화마다 다시 읽음)
- 잘 쓰는 법은 Day 2 'CLAUDE.md와 Skill'에서 — 여기선 "규칙 문서를 먼저 읽는다"는 원리만

## 실습 — 첫 프롬프트

1. 바탕화면에 `vibe-day1` 폴더 만들기 — 이 폴더가 "프로젝트" (바탕화면 자체가 아니라 그 안 폴더)
2. 폴더 주소창 클릭 → `powershell` → Enter → `claude` 입력
3. 아래 프롬프트 전송
4. 생긴 `index.html` 을 더블클릭해 브라우저로 열기
5. 화면이 뜨면 성공! (강사 화면과 달라도 정상 — 샘플링)
6. `Win+Shift+S` 로 캡처해 모아두기

### EXAMPLE PROMPT

내 이름과 오늘 날짜, "나의 첫 바이브코딩"이라는 제목이 있는  
한 페이지짜리 HTML을 만들어줘. 배경은 밝은 색, 글자는 화면 가운데 크게.

## 개발환경 — 이 장을 마치면

- [ ] 터미널에 `claude --version` 을 치면 버전 번호가 나온다
- [ ] 내가 만든 `index.html` 이 브라우저에서 열린다
- [ ] "프롬프트가 뭐예요?"에 한 문장으로 답할 수 있다

## 개발환경 — 더 깊이 (참고 자료)

- Claude Code 설치 공식 문서
- Claude Code 퀵스타트 (첫 사용법)
- 슬래시 명령어 전체 목록
- VS Code에서 Claude Code 쓰기

## 웹의 구조

### 프론트엔드 / 백엔드 / 데이터베이스

*DAY 1 · 10:30–11:10 · 이론 · 실습*

웹 서비스는 딱 3개의 부품으로 이루어집니다. 이 큰 그림만 잡으면 훨씬 잘 시킬 수 있습니다.

(도식: 프론트엔드, 백엔드, 데이터베이스 구조 도식)

## 3개의 부품 한눈에 보기

부품	하는 일	인스타로 치면
프론트엔드	보이는 전부 — HTML·CSS·JS	피드 화면
백엔드	뒤에서 일하는 서버 — 로그인·주문 접수	좋아요 접수 서버
데이터베이스	깎다 켜도 남을 데이터 창고	기록 저장되는 곳



## 프론트엔드 3요소 — HTML · CSS · JS

집 짓는 순서와 똑같습니다 — 골조 → 인테리어 → 설비.

- HTML — 뼈대·구조. "제목·문단·버튼"처럼 의미 부여, 검색엔진이 알아보게 → 골조
- CSS — 뼈대에 옷 입히기. 색·글꼴·크기·배치 → 인테리어·도색
- JS(자바스크립트) — 기능·동작. 버튼 반응, 움직임, 입력 처리 → 전기·수도 설비

💡 [캡처: 같은 페이지 3단계 — ① HTML만 ② CSS 적용 ③ JS까지]

## HTML · CSS · JS 역할 표

요소	맡은 역할	집짓기 비유
HTML	뼈대·구조, 정보에 의미 부여	 골조
CSS	스타일 — 색·글꼴·배치	 인테리어·도색
JS	기능·동작 — 클릭·움직임	 전기·수도 설비

- 같은 페이지를 세 번에 나눠 시켜보면 각각 무엇을 바꾸는지 눈으로 체감된다

## EXAMPLE PROMPT — HTML → CSS → JS 3단계

### ① 뼈대만 (HTML)

내 자기소개 웹사이트를 HTML만으로(CSS·JS 없이) 담백하게 만들어줘.  
이름, 한 줄 소개, 좋아하는 것 3가지, 연락처만 넣어줘.

### ② 옷 입히기 (CSS)

내용은 그대로 두고, 여기에 CSS만 더해서 더 멋지게 꾸며줘. 색·글꼴·여백을 넣어 보기 좋게.

### ③ 움직임 (JS)

이제 다크모드/라이트모드 전환 버튼과 그 기능을 추가해줘. 버튼을 누르면 화면 색이 바뀌게.

## 데이터베이스는 사실 엑셀과 같습니다

엑셀에서	데이터베이스에서
파일 (.xlsx)	데이터베이스
시트	테이블
행 (가로 한 줄)	레코드 — 데이터 한 건
열 (세로 한 칸)	컬럼 — 항목 이름

- 방금 만든 페이지가 바로 프론트엔드 — 오늘 만드는 건 전부 프론트엔드만으로 완성
- 백엔드·DB가 "필요해지는 순간"은 Day 2에 직접 만난다

## "저장"은 어떻게 — 왕복 4단계

- "주문하기" 누르면: ① 화면이 서버에 요청 → ② 서버가 DB에 기록 → ③ DB가 결과 반환 → ④ 서버가 화면에 "완료"
- 서버·DB 없이 화면에만 저장하면? 새로고침하면 지워지는 칠판
- 구조를 알면 "저장 안 돼, 고쳐줘" 대신 "데이터가 저장되게 바꿔줘"라고 짚어 말할 수 있다

# 코드를 읽는 눈 — 딱 4단어

목표는 코드를 쓰는 게 아니라 Claude의 계획·보고를 **알아듣는 것**.

단어	쉬운 뜻	예
변수	값에 붙인 이름표	<code>price = 3000</code>
조건문	"만약 ~라면"	100점이면 "축하" (if)
반복문	"각각에 대해"	상품 100개 하나씩 그리기 (for)
함수	이름 붙인 작업 묶음	"점수계산()"

- 읽는 눈이 생기면 지시가 달라진다 — "조건문 기준 금액을 5만 원으로 바꿔줘"

## 실습 — 내 페이지 해부하기

1. 브라우저 **F12** (노트북 **Fn+F12**) → Elements 탭에서 내 문장 찾기 = HTML, 오른쪽 Styles = CSS
2. "이 페이지에서 프론트엔드는 어디까지야? 백엔드가 필요한 기능 3개 제안해줘"
3. "변수·조건문·함수를 하나씩 찾아 비개발자에게 설명하듯 한글로 번역해줘"
4. "그런 코드가 없다"고 하면 → "간단한 예시를 하나 추가해서 보여줘"

## 웹의 구조 — 이 장을 마치면

- [ ] 프론트엔드·백엔드·데이터베이스를 식당에 비유해 설명할 수 있다
- [ ] F12를 눌러 내 페이지의 HTML을 직접 눈으로 찾아봤다

**참고 자료** — 웹은 어떻게 동작하는가 (MDN 한국어) / JavaScript 첫걸음 (MDN 한국어)



## 프롬프트 엔지니어링 + 토큰 절약법

*DAY 1 · 11:10–12:00 · 이론 · 실습*

같은 AI라도 시키는 사람에 따라 결과가 완전히 달라집니다. 잘 시키는 법을 체화합니다.

(도식: 좋은 프롬프트의 4요소(PTCF) 도식)

## 좋은 프롬프트의 4요소 — PTCF

요소	스스로 묻는 질문	예시
P 역할	누구로서?	"너는 소개 페이지 잘 만드는 웹 디자이너야"
T 할 일	무엇을?	"한 장짜리 소개 페이지 만들어줘"
C 맥락	대상·목적·조건?	"채용 담당자 / 갈색·크림색만 / 한 화면"
F 형식	어떤 산출물로?	"index.html 한 파일, 브라우저로 열어줘"

- '목표'와 '제약'은 모두 C(맥락)에 담습니다 — 이 둘이 결과를 가장 크게 바꿉니다

## 왜 구체적이어야 하나 + 토큰 절약

- AI는 마음을 못 읽는다 — 안 말한 부분은 세상의 평균으로 채운다
- 큰 것을 시킬 땐 첫 줄에 "단계적으로 개발할 예정" 선언 후 지금 할 일 하나만
- 첫 결과가 아쉬운 건 정상 — 다시 시키지 말고 아쉬운 부분만 짚기
- 토큰 절약 3가지 — ① 주제 바뀌면 `/clear` ② 관련 파일만 ③ 한 번에 하나씩

## 외워둘 만능 프롬프트 6개

상황	이렇게 말한다
에러가 났을 때	"에러 나는데 고쳐줘" + [스크린샷]
방금 게 마음에 안 들 때	"방금 한 거 되돌려줘" (같은 대화 안)
새 대화 시작 직후	"이 프로젝트 구조랑 현재 상태 정리해줘"
다음 작업 넘어가기 전	"페이지 전체 점검해줘, 깨진 거 없는지"
대화가 길어져 느릴 때	/compact
큰 걸 시작할 때	"단계적으로 개발할 예정. 지금은 ○○만"

## 실습 — 나쁜 vs 좋은 프롬프트

1. 1차: "자기소개 페이지 예쁘게 만들어줘"
2. 2차: 아래 PTCF를 갖춘 프롬프트로 다시
3. 두 결과를 나란히 놓고 비교 — 어떤 요소가 가장 크게 바뀌었나

### EXAMPLE PROMPT (2차)

너는 카페 소개 페이지를 잘 만드는 웹 디자이너야. [P 역할]  
따뜻한 동네 카페를 알리는 소개 페이지를 만들어줘. [T 할 일]  
3초 안에 "따뜻한 동네 카페" 인상 / 스크롤 없이 한 화면 / 갈색·크림색만. [C 맥락]  
index.html 한 파일, 상단 로고·중앙 한 줄 소개·하단 인스타 링크 버튼. [F 형식]

## 프롬프트 — 더 깊이 (참고 자료)

- 프롬프트 엔지니어링 공식 문서 (Anthropic)
- 구글 공식 프롬프트 가이드 (PTCF 출처)
- Claude Code 활용 베스트 프랙티스

## 바이브코딩 플로우 + 깃(Git)

*DAY 1 · 13:00–13:30 · 이론*

오후 프로젝트에서 그대로 밟을 3단계 플로우와, 결과물을 인터넷에 공개하는 수단(깃)을 배웁니다.

## 모든 프로젝트가 밟는 3단계

단계	하는 일	한 줄 요약
① 기획·디자인	무엇을 왜 만들지	"누가 보나/무엇을/어떤 분위기" 세 가지
② 구현·디버깅	만들고 고친다	섹션 하나씩, 한 번에 하나만
③ 배포	인터넷에 공개	내 파일이 전 세계 URL이 된다

(도식: GitHub Pages 배포 흐름 도식)



## 깃(Git) — 겁먹지 않아도 되는 이유

- 깃 = 코드의 **세이프 포인트**. "여기까지 저장"을 남기고, 망치면 되돌린다
- 왜 Ctrl+S로 부족한가 — ① 어제 상태로 못 돌아감 ② 컴퓨터 고장 나면 사라짐 ③ 배포하려면 코드가 클라우드에 있어야 함

## 용어 4개 — 택배로 기억하기

용어	뜻	택배 비유
저장소	프로젝트가 사는 방 (GitHub)	택배 상자
커밋	저장 지점 만들기	포장.송장
푸시	클라우드에 올리기	발송
배포	누구나 보게 공개	배달 완료

- 깃허브 = 코드계의 구글드라이브 / GitHub Pages = 올린 HTML을 공짜 웹사이트로 (오늘의 배포 수단)

## 깃 vs 깃허브 — 헛갈리지 않게

	깃(Git)	깃허브(GitHub)
정체	내 컴퓨터의 세이브 관리 도구	인터넷에 백업·공유하는 저장소
어디에	내 컴퓨터 안	인터넷
게임 비유	게임의 저장 기능	클라우드 세이브
없으면	되돌리기 안 됨	고장 나면 코드 사라짐

- 깃으로 세이브(커밋) → 깃허브로 올린다(푸시). 이 두 단계가 한 세트

# 명령 두 개만 더 — 클론과 풀

용어	뜻	비유
커밋	세이브 지점 만들기	포장·송장
푸시	클라우드에 올리기	발송
클론(clone)	저장소 통째로 내려받기	상자 복제해 책상에
풀(pull)	최신 내용만 가져와 반영	바뀐 부분만 배달

## 비개발자는 명령을 외울 필요가 없다

- 위 용어들은 Claude가 하는 말을 알아듣기 위한 것 — 내가 타이핑하려고 외우는 게 아니다
- 말로 시키면 Claude가 대신 처리

### EXAMPLE PROMPT (깃 시작 / 커밋)

이 폴더에 깃을 초기화해줘. 앞으로 여기서 저장 지점을 만들 수 있게.

지금까지 만든 걸 커밋해줘. 커밋 메시지는 지금 상태에 맞게 적당히 지어줘.

## EXAMPLE PROMPT (이력 보기 / 되돌리기)

지금까지의 커밋 이력을 보기 쉽게 보여줘.

방금 것 말고, 그 이전 커밋 상태로 되돌려줘.

💡 커밋은 자주 — 제대로 작동하는 지점마다 세이브하면 망가졌을 때 가장 가까운 멀쩡한 지점으로 돌아올 수 있습니다. 깃허브를 쓰려면 **GitHub 계정(무료)**이 필요합니다.

## 깃 — 더 깊이 (참고 자료)

- GitHub Pages란? (공식 문서, 한국어)
- Git 공식 사이트 (다운로드)

## 프로젝트 1 · 나만의 프로필 페이지

*DAY 1 · 13:30–15:00 · 프로젝트 1*

기획 → 구현 → 배포의 풀사이클. 끝나면 세상 어디서든 열리는 내 URL이 생깁니다.



## 진행 순서

단계	시간	하는 일
① 기획	15분	구성부터 Claude와 함께 정한다
② 구현	40분	섹션 하나씩 요청 (전부보다 "소개 섹션만")
③ 다듬기	15분	구체적으로 — "줄 간격을 넓혀줘"
④ 배포	20분	배포 순서를 그대로 따라간다

### EXAMPLE PROMPT (기획)

나를 소개하는 프로필 페이지를 만들고 싶어. 만들기 전에 먼저, 어떤 섹션(소개, 경력, 취미, 연락처 등)으로 구성하면 좋을지 목록으로 제안해줘. 분위기는 "차분하고 신뢰감 있게".

## EXAMPLE PROMPT (구현 — 빈칸만 채우기)

P(역할): 너는 개인 소개 페이지를 잘 만드는 웹 디자이너야.

T(할 일): [내 이름]을 소개하는 한 장짜리 페이지를 만들어줘.

C(맥락):

- 보는 사람: [친구/채용 담당자/잠재 고객]
- 목적: [나를 알리기/포트폴리오/연락 받기]
- 구성: 큰 제목 + 한 줄 소개 → 특징 3가지 → 연락 방법
- 분위기: [차분한/밝은/전문적인] 톤, 포인트 색은 [좋아하는 색]

F(형식):

- **index.html** 파일 하나로 (HTML+CSS+JS만)
- 휴대폰에서도 안 깨지게
- 완성되면 브라우저에서 바로 열어줘

## 배포 순서 — GitHub Pages (클릭만으로)

1. 확인 2가지 — 저장소는 **Public**, 첫 화면 파일명은 정확히 `index.html`
2. github.com → + → New repository → 이름(영문 소문자) → Create
3. uploading an existing file → 폴더 안 파일 전부 드래그 → Commit changes
4. Settings → Pages → Source: Deploy from a branch, Branch: main + / (root) → Save
5. 새로고침하면 "Your site is live at..." — 주소는 `https://아이디.github.io/저장소이름/`
6. 내 폰으로도 열어보기 — 진짜 인터넷에 있다는 뜻

## 프로필 — 이 장을 마치면

- [ ] 내 프로필 페이지가 `https://아이디.github.io/...`로 열린다
- [ ] 폰으로도 열어봤다
- [ ] 옆 사람과 URL을 주고받아 구경했다

💡 수정은 쉽습니다 — 저장소에서 Add file → Upload files로 같은 이름 파일을 다시 올리면 덮어쓰기되고 1~2분 뒤 반영. 첫 배포만 최대 10분.

## 프로필 — 잘 안 될 때

증상	해결
404 페이지	첫 화면 파일명이 <code>index.html</code> 이 아님 → "index.html로 바꿔줘"
고쳤는데 그대로	몇 분 뒤 <b>Ctrl+F5</b> 강력 새로고침
Pages 메뉴 안 켜짐	저장소가 Private → Public으로 변경
모양이 깨짐	"CSS 파일 경로 맞는지 확인해줘, 대소문자까지"

**참고 자료** — [GitHub Pages 퀵스타트](#) / [Pages 사이트 만들기 단계별 안내](#)

## 프로젝트 2 · 고전게임 만들기

### 디버깅 사이클 체득

*DAY 1 · 15:10–16:40 · 프로젝트 2*

목표는 완성이 아니라 버그를 만나고 고치는 경험입니다.

(도식: 디버깅 순환 도식)

## 에러를 만나면 — F12 → Console → 캡처

- 브라우저 `F12` → Console 탭 → 빨간 글씨. 개발자도구 마스터할 필요 없다
- 최고의 에러 신고 — `Win+Shift+S` 로 캡처해 붙여넣고 "이 에러 고쳐줘" (Claude가 이미지에서 읽어냄)
- 에러가 안 보일 땐 — 뭘 했고 / 뭐가 나왔고 / 뭘 원하는지 세 가지 (아래 템플릿)

# 디버깅 3원칙 — 3일 내내 쓴다

원칙	이렇게	왜
① 에러는 그대로	복사·캡처로 전문, 요약 금지	에러 전문이 최고 단서
② 기대 vs 실제	"기대는 A인데 실제로는 B"	차이를 정확히 좁힘
③ 한 번에 하나	하나 고치고 확인, 그다음	동시에 고치면 원인 모름
+ 3번 실패	"다른 방식으로" 또는 처음부터	같은 길은 같은 곳에 도착

💡 **마지막 탈출구 — 새 폴더에서 다시 시작해도 됩니다.** 점점 꼬이면 새로 만드는 게 빠를 때가 있습니다. 실패가 아닙니다.



## 구체적일수록 결과가 좋아진다 — 사과게임

이번 실습은 사과게임으로 "구체적으로 지시하는 연습". 완성이 목적이 아니라 규칙을 또박또박 설명할수록 결과가 어떻게 달라지는지 느끼는 것.

- 그냥 "게임 만들어줘"면 AI가 빈칸을 마음대로 채운다 — 상상과 전혀 다른 게 나옴
- 규칙·화면·조작법을 풀어주면 내 머릿속에 훨씬 가깝게 만든다 — 바이트코딩의 가장 중요한 기술

## 사과게임이란

- 화면에 1~9 숫자가 적힌 사과들이 직사각형 격자로 뱅뱅하게 놓여 있다
- 마우스로 드래그해 사각형 범위를 지정, 그 안의 사과를 한꺼번에 선택
- 선택한 숫자의 합이 정확히 10이면 사라지고 점수↑ (10 아니면 아무 일 없음)
- 제한 시간(예: 2분) 안에 최대한 많이 없애기

💡 **먼저 진짜 게임을 3분만 해보세요** ([gamesaien.com](http://gamesaien.com) 사과게임). 직접 해본 게임은 규칙 설명이 훨씬 쉽습니다.

## 실습 — 사과게임 만들기

1. 먼저 진짜 사과게임을 해보며 규칙 익히기
2. "만들어줘"가 아니라 "계획부터 세워줘" — 계획을 읽으면 빠진 부분이 보임
3. 계획이 좋으면 "이 계획대로 만들어줘"
4. 완성되면 진짜 게임과 나란히 켜고 다른 점을 하나씩 개선
5. 버그는 버그 신고 템플릿으로

## EXAMPLE PROMPT (계획부터 — plan-first)

사과 게임을 만들 계획을 세워보자. [https://www.gamesaien.com/game/fruit_box_a](https://www.gamesaien.com/game/fruit_box_a) 와 같은 게임이야.

규칙:

- 화면에 1~9 숫자 사과들이 직사각형 격자로 뿔뿔하게 놓여 있어.
- 마우스로 드래그해 사각형 범위를 지정하면 그 안의 사과들이 선택돼.
- 선택한 숫자의 합이 정확히 10이면 사라지고 점수가 올라가. 10이 아니면 아무 일도 없어.
- 제한 시간 2분 안에 최대한 많이 없애는 게 목표야.

사과는 별도 이미지 없이 원 배경 위에 1~9 숫자를 그려서 표현해줘.

완성되면 `index.html` 파일 하나로, 더블클릭해 브라우저로 바로 열 수 있게.

일단 코드부터 짜지 말고, 계획부터 세워줘.

## 왜 "계획부터"일까

💡 바로 만들게 하면 잘못된 방향으로 한참 가버립니다. 계획을 먼저 읽어보면 **빠진 규칙**(예: 시간이 끝나면 어떻게 되는지)이 보이고, 코드 나오기 전에 바로잡을 수 있습니다.

### EXAMPLE PROMPT (최종 산출물 형태)

완성되면 `index.html` 파일 하나로 만들어줘.  
따로 서버를 켜지 않아도, 그 파일을 더블클릭하면 브라우저에서 바로 열려서 플레이할 수 있게 해줘.

## EXAMPLE PROMPT (버그 신고 템플릿)

버그가 있어.  
기대한 것: 합이 10인 사과를 드래그하면 사라져야 해.  
실제 동작: 드래그해도 아무 반응이 없어.  
에러 메시지: (F12 콘솔에 뜬 빨간 글씨를 그대로 붙여넣기)  
이것만 고쳐줘. 다른 부분은 건드리지 마.

## 진짜 게임과 비교하며 다듬기

창 두 개로 나란히 켜고 ****"진짜는 이런데 내 건 이래"***를 **하나씩** 짚습니다. 한 번에 하나가 핵심.

### 세부 튜닝 예시

1. 드래그하는 동안 선택 영역이 안 보임 → 미리보기 추가
2. 사과가 너무 적거나 큼 → 개수·크기 조정
3. 사라진 자리에 새 사과가 채워짐 → 원래 규칙대로 안 채우게

## EXAMPLE PROMPT (드래그 영역 미리보기 / 개수·크기)

지금은 마우스로 드래그해도 어디를 고르는지 안 보여.  
진짜 사과게임처럼, 드래그하는 동안 선택 중인 사각형 영역이 반투명한 박스로 보이게 해줘.  
이 부분만 고치고 다른 건 건드리지 마.

지금 사과가 너무 크고 개수가 적어서 싱거워.  
진짜 게임처럼 가로 **17**칸, 세로 **10**칸 정도로 촘촘하게 배치하고, 크기도 그에 맞게 줄여줘.  
이 부분만 고쳐줘.



## EXAMPLE PROMPT (사라진 자리 안 채우기 + 드래그 다듬기)

두 가지만 고쳐줘.

1) 합이 10이 되어 사과가 사라진 자리에 새 사과가 다시 채워지고 있어. 원래 규칙대로 빈칸으로 뒤.

2) 드래그가 어색해 - 살짝만 움직여도 똑똑 끊겨. 부드럽게 이어지도록 다듬어줘.

한 번에 하나씩 확인할 거니까, 1번부터 고쳐주고 결과를 알려줘.

## 개발자 도구 아주 살짝

- 브라우저 **F12** → 개발자 도구. 마스터할 필요 없고 딱 하나만
- 왼쪽 위 **요소 선택기(화살표)** 누른 뒤 화면 특정 부분 클릭 → 어떤 요소인지 정보 표시
- "이 부분이 이상해"라고 할 때 요소 선택기로 짚거나 **Win+Shift+S** 로 캡처해 붙이면 훨씬 정확

💡 [캡처: 크롬 개발자 도구(F12) — 요소 선택기로 사과 하나 클릭한 화면]

## 고전게임 — 이 장을 마치면

- [ ] 내 게임이 실제로 플레이된다 (완성도는 아직 중요하지 않음)
- [ ] 버그를 최소 한 번 만나 템플릿으로 신고해 고쳐봤다
- [ ] "기대한 것 vs 실제 동작"으로 말하는 요령이 입에 붙기 시작

**참고 자료** — 브라우저 개발자도구란?(MDN) / Console 사용법(Chrome 공식)

## Day 1 회고

*DAY 1 · 16:40–17:00 · 회고*

오늘 만든 것을 자랑하고, 막혔던 지점을 함께 정리하며 하루를 닫습니다.

### 함께 하기

- 배포 URL 릴레이 — 서로의 프로필 페이지 방문 (아침만 해도 없던 것)
- 가장 크게 막혔던 지점과 해결법 공유 — 남의 막힘이 내일의 내 해결책

## Day 1 — 스스로 점검

3개에 답할 수 있으면 오늘은 성공:

- 프론트엔드 / 백엔드 / 데이터베이스는 각각 무슨 일을 하나?
- 좋은 프롬프트의 4요소(PTCF)는?
- "배포"란 무엇이고, 나는 오늘 어떻게 배포했나?

💡 Day 2 예고 — 내일은 날씨·유튜브 같은 바깥 세상의 데이터를 내 서비스로 끌어옵니다 (API).

## DAY 2 — 연결

*API로 "진짜 서비스"를 만든다*

## 연결 — API로 "진짜 서비스"를

### DAY 2

오늘의 목표: 외부 데이터·기능(API)을 연결하고, 웹을 넘어 크롬 익스텐션 형태까지.

산출물 — 크롬 익스텐션 1개 + 데이터 시각화 AI 서비스 1개 + 테스트 결제 성공

어제는 "내가 쓴 내용"만, 오늘은 **바깥 세상의 살아있는 데이터**를 끌어옵니다.

## DAY 2 시간표 (1/2)

시간	구분	세션
09:00-09:30	복습	Day 1 복습 퀴즈
09:30-10:10	이론+실습	CLAUDE.md와 Skill
10:10-10:40	이론	API란 무엇인가
10:40-11:00	완충	랜덤 이미지 API 미술관



## DAY 2 시간표 (2/2)

시간	구분	세션
11:00-12:00	프로젝트	프로젝트 3 · 구글 API 크롬 익스텐션
13:00-14:50	프로젝트	프로젝트 4 · 공공 API 시각화 + AI
15:00-16:40	따라하기	쇼핑몰 + 결제 연결
16:40-17:00	회고	Day 2 회고

## Day 1 복습 퀴즈

DAY 2 · 09:00–09:30 · 복습

어제 배운 내용을 복습하고, 막힌 부분을 해소한 뒤 시작합니다.

### 스스로 점검

- 프론트엔드 / 백엔드 / DB는 각각 무슨 일을 하나? (식당 비유)
- 좋은 프롬프트의 4요소(PTCF)는? 내 프롬프트엔 몇 개 있었나?
- "배포"란? 내 프로필 페이지는 지금도 열리나? (폰으로 확인)
- 버그를 만났을 때 AI에게 어떻게 말해야 하나?

## CLAUDE.md와 Skill

### AI에게 규칙을 기억시키기

DAY 2 · 09:30–10:10 · 이론 · 실습

어제 매번 반복했던 "한국어로 답해줘", "한 파일로 해줘" — 그 반복을 파일 하나로 없앱니다.

## CLAUDE.md — 기억이 리셋되는 비서에게

- 클로드 코드는 새 대화마다 기억이 리셋 — 책상 위 업무 지침서가 CLAUDE.md
- 프로젝트 폴더에 두면 자동으로 먼저 읽힘 — `/clear` 후에도 적용, 토큰 절약과도 궁합
- 위치 중요 — 반드시 프로젝트 맨 위(루트) 폴더. 하위 폴더면 못 찾음
- 쉬운 생성법 `/init` — 분석해 알아서 만들어줌, 그 위에 내 규칙을 보탬
- 작성 팁 — ① 간결하게 ② 구체적으로 ③ 처음부터 완벽 말고 점점 키움
- 업데이트 — 새 규칙 생겼을 때, AI가 같은 실수 반복할 때

## Skill — 반복 작업을 명령어 하나로

- Skill = 자주 하는 절차를 등록해 /이름 한 줄로 실행 (엑셀 매크로와 비슷)
- 만드는 법도 말로 — "○○하는 스킬을 만들어줘"
- 좋은 대상 — ① 반복되고 ② 순서가 정해지고 ③ 자주 하는 작업. "코드 잘 짜줘" 같은 막연한 건 스킬이 아님

## 실습 — 오늘 쓸 CLAUDE.md 만들기

1. 바탕화면에 `vibe-day2` 폴더 새로 (어제 프로필 폴더 아님)
2. 그 안에 `CLAUDE.md` 만들기 (Claude에게 시켜도 됨)
3. 아래 예시 참고해 내 규칙 적기
4. 새 대화 열어 아무 요청, 규칙이 자동 적용되는지 확인

### CLAUDE.md 예시

```
프로젝트 규칙
- 답변은 한국어로, 설명은 초보자 눈높이로
- HTML/CSS/JS는 파일 하나에 모아서 작성
- 코드를 바꾸면 무엇을 왜 바꿨는지 한 줄로 알려주기
- API 키는 절대 코드에 직접 적지 않기
```

## CLAUDE.md — 이 장을 마치면

- [ ] 내 프로젝트 폴더에 CLAUDE.md가 들어 있다
- [ ] 새 대화를 열어도 내가 적은 규칙이 자동으로 지켜지는 걸 확인했다

**참고 자료** — CLAUDE.md(메모리) 공식 문서 / Skill 공식 문서

## API란 무엇인가

DAY 2 · 10:10–10:40 · 이론

오늘 하루 종일 쓸 단 하나의 개념 — "정해진 형식으로 요청하면, 데이터가 온다."

(도식: API 요청과 응답 도식)



## API — 핵심 내용

- API = Application Programming Interface. "응용 프로그램끼리 대화하는 접속 창구"
- 흐름은 항상 같다 — 요청("서울 미세먼지 주세요") → 응답("여기 있습니다")
- 응답은 주로 JSON — 중괄호 `{ }` 로 정리된 데이터 꾸러미 (읽는 건 AI가 해줌)
- API 키 = "요청할 자격이 있어요" 증명하는 열쇠. 내 명의로 사용량 기록
- 무료 API — 공공데이터포털(data.go.kr), Google Cloud 콘솔

## ! 키는 비밀번호다 — 오늘 가장 중요한 안전 수칙

- 어제 깃허브 올리는 법을 배웠기에 오늘이 가장 위험 — 키가 적힌 코드를 올리면 요금 폭탄
- 안전 수칙 ① — "API 키가 코드에 노출되지 않게 해줘"라고 요청
- 안전 수칙 ② — 깃허브에 올리기 전, 키가 든 파일이 포함됐는지 반드시 확인
- 오늘 실습은 연습용 무료 키라 코드에 넣되 깃허브엔 안 올리는 게 규칙. 실제 서비스는 키를 서버에 숨김(백엔드가 필요한 이유)

참고 자료 — 공공데이터포털(data.go.kr) / Google Cloud 콘솔

## 랜덤 이미지 API 미술관

### 키 없이 첫 성공

DAY 2 · 10:40-11:00 · 완충 실습

키 발급도 가입도 없이 API를 붙여 미술관 사이트를 완성, "API로 데이터를 받아 화면에 뿌리는" 감을 익힙니다.

## 왜 이걸 먼저 하나 — 완충(warm-up)

- 곧 배울 구글·공공데이터 API는 가입·키 발급·신청이 많아 시작부터 막히기 쉬움 — 그 앞에서 지치면 재미있는 부분에 도달 전 힘 빠짐
- 키가 전혀 필요 없는 API로 "성공"을 먼저 맛보면 뒤 준비 단계가 만만해짐
- 핵심은 준비물이 아니라 흐름 — 주소로 요청 → 데이터 응답 → 화면에 뿌림

## picsum.photos — 키가 필요 없는 API

- 주소 하나만 부르면 랜덤 사진을 돌려주는 무료 이미지 API. 가입도 키도 없음 — 주소가 곧 요청
- `https://picsum.photos/300` → 가로·세로 300px 랜덤 이미지. 끝 숫자가 크기, 부를 때마다 다른 사진
- 브라우저 주소창에 직접 쳐 보세요 — 새 사진이 뜨면 그게 API가 응답하는 순간

💡 주소 뒤에 붙여 보기 — `/300/200` (가로·세로), `/300?grayscale` (흑백), `/300?blur` (흐리게). 요청을 조금씩 바꿔 보는 습관이 내일 진짜 API에도 그대로.

## 실습 — 가상 미술관 사이트

1. 새 폴더에 아래 계획 먼저 프롬프트로 뼈대 생성
2. 계획을 읽고 "좋아, 그대로 만들어줘" (마음에 안 들면 여기서 고침)
3. 브라우저로 열기 — 랜덤 사진 30장이 격자로 뜨면 1차 성공 (새로고침마다 바뀜)
4. 개선 프롬프트로 전시회 디테일 엮기

### EXAMPLE PROMPT (계획 먼저 — 미술관 뼈대)

`https://picsum.photos/300` 처럼 요청하면 `300x300` 랜덤 이미지를 돌려주는 무료 API가 있어. 키나 가입은 필요 없어. 이걸 이용해 가상 사진 전시회(온라인 미술관)처럼 보이는 웹사이트를 만들어줘. 이미지는 30장 정도를 격자(그리드)로 보기 좋게 배치하면 돼. 바로 코드부터 짜지 말고, 어떻게 만들지 계획부터 세워서 보여줘.

## EXAMPLE PROMPT (미술관 개선 ①②)

### ① 작품처럼 꾸미기

좋아. 이제 각 사진을 액자에 넣은 작품처럼 보이게 해줘.  
사진마다 아래에 작품 제목과 작가 이름을 그럴싸하게 붙이고(예: "무제 No.12 - 김바로"),  
페이지 맨 위에 미술관 이름과 한 줄 소개도 넣어줘.

### ② 움직임 넣기

마우스를 사진 위에 올리면 살짝 커지면서 제목이 도드라지게 해줘.  
사진을 클릭하면 큰 화면으로 확대돼 감상할 수 있으면 더 좋아.



[캡처: picsum 미술관 사이트 — 랜덤 사진 30장 격자, 제목·작가명]

# 미술관 — 잘 안 될 때

증상	해결
빈 네모(깨진 이미지)만	인터넷 확인·새로고침 → "주소가 <code>https://picsum.photos/300</code> 형식 맞는지 확인해줘"
30장이 전부 같은 사진	"사진마다 다르게 나오게 해줘" (주소를 조금씩 다르게)
세로 한 줄로만 쌓임	"사진을 여러 열의 격자로 정렬해줘"



## 미술관 — 이 장을 마치면

- [ ] 키도 가입도 없이 API로 받아온 사진 30장이 격자로 놓인 미술관이 브라우저에 떠 있다
- [ ] "주소로 요청하면 데이터가 오고 화면에 뿌린다"는 API 한 바퀴를 직접 겪었다
- [ ] 다음 세션부터 키가 필요한 진짜 API로 넘어갈 준비가 됐다

## 프로젝트 3 · 구글 API 크롬 익스텐션

*DAY 2 · 11:00–12:00 · 프로젝트 3*

웹페이지가 아니라, 크롬에 실제로 설치해서 쓰는 프로그램을 만듭니다.

(도식: 크롬 익스텐션 구조 — 파일 3개짜리 폴더가 곧 익스텐션. 셋 다 Claude가 만듦)

## 크롬 익스텐션 — 핵심 내용

- 크롬 익스텐션 = 크롬에 설치하는 작은 프로그램 (광고 차단기·번역기가 전부 익스텐션)
- 예시 — YouTube Data API로 "채널명 넣으면 구독자·최근 영상", Google Maps API로 "동네 즐겨찾기 지도"
- 스토어 등록 없이 내 크롬엔 즉시 설치 — "개발자 모드 로드"

## 실습 순서

1. 새 폴더 만들고 아래 프롬프트로 익스텐션 생성
2. 크롬 주소창 `chrome://extensions` → 우상단 개발자 모드 켜기
3. "압축해제된 확장 프로그램을 로드합니다" → 내 폴더 (manifest.json이 바로 보이는 폴더)
4. 우상단 퍼즐 아이콘에서 내 익스텐션 클릭 — 실행!
5. 구글 API 실제 데이터가 팝업에 뜨면 완성

## EXAMPLE PROMPT

크롬 익스텐션을 만들어줘. 아이콘을 누르면 작은 팝업이 뜨고,  
유튜브 채널 이름을 검색하면 **YouTube Data API**로  
구독자 수와 최근 영상 목록을 보여주는 기능이야.  
**API** 키는 발급받아 뒀어. 키를 넣을 위치만 알려주면 내가 직접 넣을게.

## 익스텐션 — 잘 안 될 때

증상	해결
빨간 "오류" 버튼	눌러 메시지 복사 → "익스텐션 로드가 안 돼. 이 에러 고쳐줘" + 붙여넣기
고쳤는데 그대로	<code>chrome://extensions</code> 에서 카드의 <b>새로고침</b> (🔄) 눌러야 반영
팝업이 하얗게만	팝업 우클릭 → 검사 → Console 빨간 글씨 캡처 → "빈 화면으로 떠. 이 에러 고쳐줘"

이 장을 마치면 — [ ] 퍼즐 아이콘 안에 내 익스텐션 [ ] 클릭하면 구글 API 진짜 데이터가 팝업에

## 프로젝트 4 · 공공 API 시각화 + AI

*DAY 2 · 13:00–14:50 · 프로젝트 4*

오후에 각자 완주하는 프로젝트는 이것 하나. 서두르지 말고 체크포인트를 하나씩.

(도식: 공공 API 시각화 AI 서비스 구조 도식)

## 공공 API 시각화 — 핵심 내용

- 공공데이터포털(data.go.kr) = 정부 데이터 창고 (날씨·미세먼지·버스 도착 무료 API)
- "AI가 데이터 기반으로 답한다" = 질문과 함께 받아온 데이터를 AI에게 주며 "이 데이터로 답해줘"
- 페이지 안의 AI는 클로드 코드와 별개 — AI API 키가 따로 필요 (수업에서 키와 함께 안내)
- 어제·오늘 배운 게 전부 조립 — 프론트엔드(Day1) + API(오전) + AI

## 단계별 체크포인트 — 순서대로 하나씩

1. API 응답(JSON)이 F12 콘솔에 찍힌다 — 여기까지가 절반
2. 받아온 숫자로 차트가 화면에 그려진다
3. 질문을 입력하면 데이터를 근거로 답이 온다

### EXAMPLE PROMPT (시작)

공공데이터포털의 미세먼지 API를 쓰는 웹페이지를 만들자.  
단계별로 가자. 먼저 API를 호출해서 응답 데이터가 브라우저 콘솔에 찍히는 것까지만 해줘.  
인증키는 발급받아 뒀어. 넣을 위치를 알려줘.



막히면 디버깅 3원칙 — ① 에러 전문 ② 기대 vs 실제 ③ 한 번에 하나



## 공공 API — 잘 안 될 때

증상	해결
"등록되지 않은 키" 에러	일반 키와 인코딩 키 2가지 — "인코딩 키로 바꿔서 다시 호출해줘"
콘솔에 CORS 빨간 에러	브라우저가 외부 요청 막은 것(내 잘못 아님) — "CORS 우회 방법 적용해줘"
에러 없는데 데이터 안 옴	F12 → Network 탭 빨간 줄 캡처 → "요청은 가는데 데이터가 안 와. 원인 찾아줘"

이 장을 마치면 — [ ] 콘솔에서 JSON 봤다 [ ] 데이터가 차트로 [ ] 질문→데이터 근거 답변

## 쇼핑몰 + 결제 연결

### 화면 따라 함께 만들기

*DAY 2 · 15:00–16:40 · 따라하기 세션*

각자 만들기가 아니라 강사 화면을 따라 함께. 목표는 "결제 성공" 화면을 전원이 보는 것.

(도식: 결제 흐름 도식)

## 결제 — 핵심 내용

- 테스트 결제 = 결제 회사(토스페이먼츠)의 연습용 가짜 결제. 키가 `test_` 면 실제 돈은 0원
- 키 두 종류 — 클라이언트 키(`test_ck_`, 결제창 띄우기, 노출 OK) / 시크릿 키(`test_sk_`, 결제 확정, 공개 금지)
- 내 키는 `developers.tosspayments.com` → 내 개발정보에 테스트 키 2개
- 흐름 — 스타터 → 상품 목록 → 장바구니 → 결제 버튼 → 토스 결제창 → 성공 화면
- 주문 내역은 저장(DB), "진짜 결제됐는지"는 화면이 아니라 서버가 확인(백엔드)

## 테스트 카드 — 이 번호로 결제

1. 카드번호: 4330-0000-0000-0001
2. 유효기간: 아무 미래 날짜 / CVC: 아무 3자리 / 비밀번호: 앞 2자리 아무 숫자
3. 결제 성공 화면이 뜨면 — 오늘의 목표 달성 🎉
  - 스타터 파일·공용 테스트 키는 수업 시작 때 배포
  - 막히면 바로 손들기 — 결제 연동은 한 글자만 틀려도 멈춘다, 부끄러운 게 아니다

## 결제 — 잘 안 될 때

증상	해결
위젯이 안 뜸	클라이언트 키가 <code>test_ck_</code> 로 시작하는지, 앞뒤 공백 확인
위젯은 뜨는데 결제 실패	시크릿 키 확인 — 서로 바꿔 넣은 경우 많음
하얀 빈 화면	F12 → Console 빨간 글씨 캡처해 Claude에게
키 발급 안 됨	손 들기 — 공용 테스트 키로 진행

이 장을 마치면 — [ ] 테스트 카드로 "결제 성공" 봤다 [ ] 결제 확인을 왜 서버가 하는지 설명 가능

## 결제 — 더 깊이 (참고 자료)

- 토스페이먼츠 개발자센터 (가입·테스트 키 발급)
- 결제위젯 연동 가이드 (공식 문서)
- 결제 테스트하는 방법 (테스트 카드 안내)

## Day 2 회고

DAY 2 · 16:40–17:00 · 회고

하루 만에 익스텐션, 데이터 서비스, 결제까지 지나왔습니다.

### 함께 하기

- 서비스 시연 릴레이 — 익스텐션·시각화 서비스 자랑
- 테스트 결제까지 뚫린 사람 박수 🙌

### 스스로 점검

- API란? 요청·응답의 흐름을 설명할 수 있나?
- API 키를 깃허브에 올리면 왜 위험한가?
- 결제에서 백엔드·DB는 각각 왜 필요했나?

💡 내일 예고 — DB로 진짜 저장되는 앱(노션 클론)을 만들어 배포합니다. 특별한 준비물 없음.

## DAY 3 — 확장

*데이터를 저장하고, 인터넷에 공개한다*



## 확장 — 저장하고 공개한다

### DAY 3

오늘의 목표: DB를 이해하고, 진짜로 저장되는 노션 클론을 만들어 인터넷에 배포한다.

산출물 — DB에 저장되는 노션 클론 + Vercel로 공개된 진짜 인터넷 주소

"내 브라우저에만" 있던 데이터가 "모두의 저장소"로 넘어가는 하루입니다.

## DAY 3 시간표

시간	구분	세션
09:00-09:30	복습	전체 흐름 복습
09:30-10:50	이론+실습	데이터베이스 입문
11:00-12:00	따라하기	노션 클론 만들기
13:00-14:30	따라하기	노션 클론 (이어서)
14:40-15:50	이론+실습	Vercel 배포
16:00-17:00	마무리	이후 학습 로드맵

## 전체 흐름 복습

DAY 3 · 09:00–09:20 · 복습

큰 그림이 오늘 완성됩니다: 만들기 → 저장하기 → 인터넷에 공개하기.

### 지금까지 온 길

- Day 1 — 프롬프트로 만들고·고치고, GitHub Pages로 공개
- Day 2 — API로 바깥 데이터 연결, 크롬 익스텐션, 결제에서 백엔드·DB 실감
- Day 3(오늘) — DB를 배우고, 노션 클론을 만들어 Vercel로 공개

함께 하기 — 오늘 만들 노션 클론 그려보기: 페이지를 만들고·저장하고·어디서든 다시 여는 앱

## 데이터베이스 입문

### 내 데이터를 어디에 저장하나

DAY 3 · 09:30–10:50 · 이론+실습

지금까지 앱들은 데이터가 내 브라우저 안에만 갇혀 있었습니다. 진짜 서비스로 넘어가려면 데이터베이스가 필요합니다.

## 먼저, 오늘 만들 앱을 상상해 봅시다

- 잠시 뒤 만들 localStorage 할 일 앱 — 새로그침해도 남습니다(브라우저가 내 컴퓨터에 적어둠)
- 결정적 한계 — 그 데이터는 "내 브라우저에만". 다른 기기·다른 사람·다른 브라우저엔 안 보임
- localStorage는 혼자 쓰는 개인 메모장 — "친구와 같이", "어느 기기서든"을 원하면 **모두가 접근하는 한곳** = 데이터베이스

## 데이터베이스란 무엇인가

두 가지가 합쳐진 것 — 데이터를 쌓는 **공간** + 안전하게 넣고 꺼내주는 **관리 시스템**.

💡 비유하면 거대한 도서관입니다. 서가(공간)만 있으면 아수라장 — 어디에 뭘 두고 누가 빌렸는지 관리하는 ****사서(관리 시스템)****가 있어야 도서관. DB = 저장 공간 + 똑똑한 사서.

- 사서 덕에 수백만 건이 쌓여도 순식간에 찾고, 동시 요청도 안 뒤텀킴
- 우리는 "넣어줘 / 꺼내줘"라고 부탁만 — 창고를 직접 뒤질 필요 없음

# 두 종류 맛보기 — 관계형 vs 문서형

	관계형 (SQL)	문서형 (NoSQL)
생김새	엑셀 표처럼 행·열이 정해진 칸	자유로운 서류 뭉치
비유	 회계 장부	 서류철
강점	데이터끼리 관계로 연결(회원↔주문)	형식 유연, 빠른 개발
대표 예	PostgreSQL, MySQL, Supabase	MongoDB, Firebase

- 다음 세션의 Supabase는 관계형(엑셀 표 방식) — 익숙한 표 형태라 이해하기 쉬움

 [캡처: 관계형(줄 맞춘 엑셀 표) vs 문서형(제각각 서류 카드) 개념 그림]

## 데이터로 하는 일은 딱 4가지 — CRUD

동작	뜻	메모 앱에서라면
Create	새 데이터를 넣는다	새 메모를 쓴다
Read	저장된 데이터를 꺼내 본다	목록을 펼쳐 본다
Update	기존 데이터를 고친다	메모 내용을 고친다
Delete	데이터를 지운다	메모를 버린다

- 이 네 단어면 어떤 앱이든 쫓개어 시킬 수 있다 — 노션 클론도 결국 페이지에 대한 CRUD



## 실습 — 저장을 눈으로, 한계까지

1. 새 폴더에서 아래 계획 먼저 프롬프트 → 계획 읽고 실행
2. 할 일 3~4개 추가(Create), 목록이 화면에(Read)
3. **핵심** — 새로고침(F5). 방금 쓴 할 일이 그대로 남음 (localStorage 저장)
4. **한계** — 같은 주소를 다른 브라우저/폰에서 열면 텅 빈. "내 브라우저에만 있었구나"

### EXAMPLE PROMPT (계획 먼저)

바로 만들지 말고 계획부터 보여줘. 브라우저의 `localStorage`를 써서 새로고침해도 목록이 남는 간단한 할 일 앱을 HTML 파일 하나로 만들 거야. 기능은 딱 네 가지 – 할 일 추가, 목록 보기, 완료 체크(수정), 삭제. 계획이 마음에 들면 그때 만들어줘.

## DB 입문 — 이 장을 마치면

- [ ] DB가 "저장 공간 + 관리 시스템(사서)"이라는 걸 도서관에 비유해 설명할 수 있다
- [ ] 관계형·문서형의 차이를 한 문장으로, Supabase가 관계형임을 안다
- [ ] CRUD 네 가지가 무엇인지 말할 수 있다
- [ ] localStorage로 저장을 확인하고, "내 브라우저에만" 남는 한계를 직접 만나봤다



"왜 폰에선 안 보이지?"라는 답답함이 진짜 DB로 넘어가는 출발점입니다.

## 노션 클론 만들기 (따라하기)

*DAY 3 · 11:00–12:00 · 13:00–14:30 · 따라하기*

강사 화면을 따라 함께. 목표는 전원이 "페이지를 만들고 → 새로고침해도 남아 있는" 화면을 보는 것.

(도식: 노션 클론 CRUD 흐름 도식)

## 노션이 뭔가요 / 오늘 만드는 것

- 노션 = 문서·메모·할 일·표를 한 곳에서 만드는 올인원 노트 앱 (페이지 안에 페이지, 고치고 지우기)
- 진짜 노션은 기능이 많지만, 우리는 뼈대 하나만 — 페이지를 만들고 저장하는 부분
- 오늘 만드는 것 — 페이지를 만들고(C)·보고(R)·수정(U)·삭제(D) 되는 간단한 앱 = CRUD
- 메모장·가계부·일기·독서기록 전부 같은 CRUD 위에
- 회원가입·소셜 로그인은 오늘 안 함 (심화로) — CRUD 하나에만 집중

💡 앞 세션의 "데이터는 꺾다 켜도 남아야 한다"가 눈앞에서 증명 — 꺾다 다시 열어도 남는 순간이  
오늘의 하이라이트

## 오늘은 "따라하기" — 공용 환경을 받아 씀

- DB가 붙는 앱은 '서버가 도는 프로젝트'라 VS Code로 열고 터미널 명령으로 실행 → 강사 스타터 폴더로 시작
- 데이터 저장엔 Supabase가 필요한데, 각자 가입.프로젝트.테이블.키 발급은 하루로 부족
- 그래서 어제 결제처럼 — 강사가 준비한 Supabase(공용 키 + 스타터 폴더)를 받아 붙이기만
- 키는 비밀번호다 — 연습용 공용 키라 함께 써도 되지만 SNS 캡처엔 안 찍히게

💡 집에서 혼자? supabase.com 무료 가입 → 프로젝트 → 내 키 2개 붙이기. 처음부터 세팅은 심화 과정에서.

## 오늘의 순서 — 4단계

단계	무엇을	왜
① 스타터 받기	폴더·공용 키를 받아 연다	같은 출발선
② 화면(UI) 먼저	노선처럼 보이는 걸모습	붙일 자리가 생김
③ DB 연결(CRUD)	Supabase 연결·저장/불러오기	오늘의 핵심
④ 동작 확인	localhost에서 만들고 지워봄	배포 전 확인

💡 화면(②) 먼저, 저장(③) 그다음. 한 번에 시키면 AI도 헛갈림 — 작업을 쪼개는 것 자체가 오늘의 기술.

## 따라하기 — 강사 화면과 함께

1. 스타터 열기 — 폴더를 VS Code로 열고 안내된 실행 명령 입력. 빈 시작 화면 뜨면 준비 완료
2. 공용 키 붙이기 — Supabase 주소·키를 설정 파일 정해진 칸에 (앞뒤 공백 주의)
3. ② UI 먼저 — 첫 프롬프트로 화면만 (아직 저장 안 돼도 정상)
4. ③ CRUD 붙이기 — 두 번째 프롬프트로 Supabase 연결
5. ④ 확인 — 페이지 만들고 수정·삭제, 브라우저 꺾다 다시 열어 남아 있으면 목표 달성 🎉
6. 막히면 바로 손들기 — 키 한 글자·공백 하나만 틀려도 멈춤
7. [각자 5분] 완성했으면 제목 색·폰트 하나만 바꿔보기 — 내 손으로 고쳐보는 게 실력

## EXAMPLE PROMPT — 노션 클론 ② UI / ③ DB

### ② UI 먼저 (룩앤필만)

노션(<https://www.notion.com/ko>)과 비슷한 느낌의 "페이지 관리 앱" 화면을 만들어줘.  
지금은 기능은 말고 룩앤필만 – 왼쪽에 페이지 목록, 오른쪽에 내용 영역이 있는 깔끔한 화면.  
저장 기능은 다음 단계에서. 바로 만들지 말고 계획부터 세워서 보여줘.

### ③ DB 연결 (CRUD)

이제 이 앱에 데이터베이스를 연결해서, 페이지를 만들고·목록으로 보고·수정·삭제(CRUD)가 되게 해줘.  
데이터베이스는 내가 제공한 Supabase 설정(주소·키)을 그대로 사용해줘.  
나중에 Vercel 배포까지 할 예정이니 그때 문제가 안 생기는 최신 연결 방법을 인터넷으로 확인해서 적용해줘.  
한꺼번에 하지 말고, 계획부터 세워서 보여주고 단계적으로 진행하자.

[캡처: 완성된 노션 클론 화면] [캡처: Supabase 테이블에 페이지들이 행으로 쌓인 화면]



## 노션 클론 — 자주 막히는 곳

증상	해결
저장이 안 됨	Supabase 주소·키, 앞뒤 공백 확인. 두 값 바꿔 넣은 경우 많음
새로고침하면 사라짐	아직 DB 미연결, 화면에만 임시 — ③ 단계 마쳤는지 확인
하얀 빈 화면	F12 → Console 빨간 글씨 캡처해 Claude에게
저장은 되는데 목록에 안 보임	Supabase 대시보드에 데이터 있는지 먼저 → 있으면 Read 부분 문제

💡 "새로고침하면 사라진다" vs "꺼다 켜도 남는다"의 차이가 바로 DB의 존재 이유. 사라지면 실패가 아니라 신호.

## 다음은 — 진짜 인터넷에 공개

- 지금 이 앱은 내 컴퓨터(localhost)에서만 — 나만 볼 수 있는 주소
- 다음 세션에서 Vercel로 배포해 진짜 인터넷 주소로 공개 (친구가 폰에서 열게)
- ③ 프롬프트에서 "배포까지 고려해 최신 방법으로" 부탁한 이유 — 지금 잘 만들면 배포가 매끄러움

## 이 장을 마치면

- [ ] 노션처럼 페이지를 만들고·보고·수정·삭제하는 앱을 움직여봤다
- [ ] 켜다 켜도 데이터가 남는 것을 확인 — DB가 왜 필요한지 실감
- [ ] 공용 Supabase를 붙여봤고, 혼자 할 땐 무엇이 더 필요한지 안다

## Vercel 배포

### DB 붙은 앱을 인터넷에 공개

DAY 3 · 14:40–15:50 · 이론+실습

앞 세션의 노션 클론은 아직 내 컴퓨터 안에서만 열립니다. 이번엔 진짜 인터넷 주소에 올립니다.

## 지금 상태 — localhost는 나만 본다

- 노션 클론 실행하면 `localhost:3000` — localhost = 바로 이 컴퓨터. 옆 사람에게 보내도 안 열림
- Day 1의 프로필처럼 인터넷에 올릴 차례 — 그런데 이 앱은 DB가 붙어 방법이 조금 다름

## 정적 vs 동적 배포 — 왜 GitHub Pages로는 안 되나

	정적(static)	동적(dynamic)
어떤 앱	Day 1 프로필 (HTML만)	노션 클론 (DB·서버)
필요한 것	파일만 있으면 됨	요청마다 서버가 DB 읽고 씀
배포 수단	GitHub Pages	GitHub Pages 불가 → Vercel
비유	인쇄된 포스터	응대하는 사람이 있는 가게

- GitHub Pages는 완성 파일만 보여줌 — "저장하면 DB에 넣어라" 같은 일을 시킬 서버가 없음
- 그래서 동적 앱은 서버를 굴려주는 배포 서비스 = Vercel

## Vercel — GitHub만 연결하면 알아서

- Vercel(vercel.com) = GitHub 저장소를 연결하면 자동 빌드·배포, 서버까지 준비
- 무료로 시작 — 개인 프로젝트.오늘의 노션 클론은 무료 범위로 충분
- 한 번 연결하면 이후엔 GitHub에 푸시만 하면 알아서 재배포

💡 Day 1의 **커밋(포장)·푸시(발송)**가 그대로. Vercel은 발송된 택배를 받아 진열대에 올려주는 배달 대행.

# 배포 4단계 — 큰 그림

단계	하는 일	한 줄 요약
① GitHub에 올리기	폴더를 저장소로	"이 프로젝트 GitHub에 올려줘"
② Vercel 연결	GitHub로 로그인, 저장소 선택	Import
③ 환경변수 등록	Supabase 키를 Vercel에	코드 아니라 Vercel에
④ 배포·공유	Deploy 후 주소 열기	<code>...vercel.app</code> 이 내 링크

### ③ 환경변수 — 키는 코드에 넣지 않는다

Day 1의 "API 키는 환경변수로 보관"을 실제로 씁니다. 노션 클론은 DB 접속에 비밀 키가 필요한데, 코드에 적으면 안 됩니다.

- 환경변수 = 실행 때 읽어가는 바깥에 붙여둔 설정 메모. 코드는 "그 메모를 읽어" 사용
- 왜 — 코드에 박으면 GitHub 올리는 순간 키까지 공개, 남이 내 DB를 열 수 있음
- 그래서 Vercel의 Environment Variables 칸에 등록 — 코드엔 "여기서 읽어와" 표시만, 진짜 값은 Vercel이 보관

흔히 넣는 환경변수	무엇
<code>NEXT_PUBLIC_SUPABASE_URL</code>	내 Supabase 프로젝트 주소
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	Supabase 접속용 공개 키

💡 어떤 이름이 필요한지는 프로젝트마다 다름 — 외우지 말고 Claude에게 물어보기



## 막히면 Claude에게 — 절차 통째로

비개발자는 화면(GUI)만으로 충분히 배포할 수 있습니다.

### EXAMPLE PROMPT (환경변수 확인)

이 프로젝트를 Vercel에 배포하려고 해.  
내 코드를 읽고, 배포하려면 어떤 환경변수가 필요한지 이름과 함께 알려줘.  
그리고 그 값들을 각각 어디서 구하는지도 설명해줘.

### EXAMPLE PROMPT (배포 절차 안내)

이 노션 클론을 Vercel에 처음 배포하려고 해.  
GitHub에 올리는 것부터 Vercel 연결, 환경변수 등록, 배포까지  
비개발자가 따라 할 수 있게 화면 클릭 위주로 단계별로 안내해줘.



개발자는 Vercel CLI를 쓰기도 — 오늘은 웹 화면 클릭만으로 끝까지.

## 따라하기 — 노션 클론을 Vercel에

1. GitHub에 올리기 — "이 프로젝트를 GitHub에 새 저장소로 올려줘" (Day 1 그대로)
2. vercel.com → Continue with GitHub 로그인 (별도 가입 불필요)
3. Add New... → Project → 저장소 Import [캡처: Vercel 프로젝트 연결]
4. Environment Variables 칸에 이름.값(Supabase URL.키) 하나씩 → Add [캡처: 환경변수 설정]
5. Deploy 누르고 대기 — 자동 빌드.서버 올림
6. `내프로젝트.vercel.app` 주소 열고 폰으로도 — 저장했다 새로고침해도 남으면 성공 [캡처: 배포 완료 주소]
7. 옆 사람에게 주소 보내 서로의 노션 클론에 글 남기기

💡 혼자 해보기 — 나만의 기능 하나(날짜 표시, 검색창) 붙이고 커밋.푸시하면 Vercel이 자동 재배포

## 보안 — 키가 GitHub에 올라가면

- 카시크릿은 절대 코드에 직접 적지 말고 환경변수로 — 이 하나만 지켜도 대부분의 사고를 막음
- 실수로 올라가면 Supabase·구글이 감지해 자동 무효화하기도 → 앱이 갑자기 멈추는 원인
- 이미 올라갔다면 — "이 키를 코드에서 빼고 환경변수로 바꿔줘. 새 키로 교체하는 방법도 알려줘"

## 정리 — 이제 정적·동적 둘 다 배포 가능

	Day 1 — 정적	Day 3 — 동적
배포 수단	GitHub Pages	Vercel
결과물	프로필 페이지 (파일만)	노션 클론 (DB 붙은 앱)
주소 모양	<code>아이디.github.io/...</code>	<code>...vercel.app</code>

- 파일뿐인 페이지는 GitHub Pages, DB·서버 앱은 Vercel — 성격에 맞춰 고르면 됨
- 기획 → 구현 → 디버깅 → 배포의 3일 전체 흐름이 정적·동적 양쪽에서 완성

## Vercel — 이 장을 마치면

- [ ] 정적(GitHub Pages)·동적(Vercel) 배포가 왜 다른지 한 문장으로
- [ ] 노션 클론이 `...vercel.app` 으로 열리고 폰으로도 접속된다
- [ ] Supabase 키를 코드가 아니라 Vercel 환경변수에 등록했다
- [ ] 키를 코드에 넣으면 왜 위험한지 알고, 옆 사람과 링크를 주고받아봤다

**참고 자료** — Vercel 시작하기 / 환경변수 설정하기 (공식 문서)

# 이후 학습 로드맵

DAY 3 · 16:40–17:00 · 마무리

3일이 끝나도 혼자서 계속 만들 수 있는 길을 챙겨갑니다.

상황	이렇게 한다
만들 것이 떠오르면	기획 3문항(누가/기능 하나/형태)부터
배포하고 싶으면	GitHub Pages는 계속 무료
데이터가 필요하면	data.go.kr, Google Cloud — "이 데이터 + 이 화면"
막히면	디버깅 3원칙 + 부록 「응급처치」
분석을 붙이고 싶으면	부록 「MS Clarity」 설치

## 부록

참고 자료·응급처치·심화

## 수강 전 준비 체크리스트

### 부록

아래는 수업 전날까지 완료. 특히 공공데이터포털은 승인에 시간이 걸리니 꼭 미리!

- [ ] GitHub 계정 (github.com) — Day 1 오후 배포용
- [ ] Google 계정 + Google Cloud 콘솔 로그인 확인 — Day 2 API 키 발급용
- [ ] 공공데이터포털(data.go.kr) 가입 + 관심 API 활용신청 미리 (승인 시간 걸림)
- [ ] Claude 유료 구독(Pro 이상) — 무료론 클로드 코드 불가
- [ ] VS Code / Node.js LTS / Git / Python 설치 (부록 「설치 상세 가이드」)
- [ ] Claude Code 설치 — 주력 도구 (설치·환경변수는 상세 가이드)

💡 막히는 항목이 있어도 첫날 아침에 함께 해결. 단 공공데이터포털 활용신청만은 꼭 미리.



## 설치 상세 가이드 — 도구 4종

### 부록

이름만 봤던 도구들을 실제로 설치하는 화면 단위 절차. 막히면 그 도구의 **버전 확인 명령**부터.

### 시작 전에 — "터미널"이란?

- 컴퓨터에 글자로 명령을 내리는 검은 창 (설치 확인용)
- 윈도우: 시작 → terminal/PowerShell / 맥: command+space → terminal

💡 버전 확인 시 숫자(`v22.11.0`)면 성공, "not recognized"면 미설치이거나 **터미널을 새로 켜야 함**.

## ① Node.js — 웹·클로드 코드의 기본 엔진

클로드 코드를 설치하려면 Node.js가 먼저 있어야 합니다.

1. <https://nodejs.org> → "LTS" 버튼 (안정 버전) [캡처: LTS 버튼 강조]
2. 설치 파일 더블클릭 → Next 계속 → Install/Finish
3. "Tools for Native Modules" 나와도 기본값으로 Next
4. 버전 확인 — 터미널 새로 열고 아래 [캡처: node -v, npm -v 결과]

명령	확인	정상 예시
<code>node -v</code>	Node.js 버전	<code>v22.11.0</code>
<code>npm -v</code>	npm 버전	<code>10.9.0</code>

 `npm`은 Node.js 설치 시 자동으로 함께 깔립니다.

## ② Git — 코드 저장·배포 도구

Day 1 오후 배포에 씁니다. 미리 깔아두세요.

1. <https://git-scm.com/downloads> → 본인 OS 클릭 → 다운로드 [캡처: git-scm 다운로드]
2. 설치(윈도우) — 옵션 화면 전부 기본값 Next → Install
3. 설치(맥) — 보통 이미 있음. 버전 확인되면 추가 설치 불필요
4. 버전 확인 — 터미널에서 `git --version` → `git version 2.47.0` 처럼 나오면 성공

💡 윈도우 설치 중 "기본 에디터 선택"이 나와도 기본값으로 두고 Next.

### ③ Python — 체크박스 하나가 핵심

설치 자체보다 체크박스 하나가 더 중요합니다.

1. <https://www.python.org/downloads/> → 노란 "Download Python 3.x" [캡처: python.org]
2. **가장 중요** — 설치 첫 화면 아래 "Add Python to PATH" 반드시 먼저 체크 [캡처: 체크한 화면]
3. 체크 확인 후 "Install Now" → Close
4. 버전 확인 — `python --version` → Python 3.13.1 (맥은 `python3 --version`)

💡 깜빡했다면? 설치 파일 다시 실행 → "Modify" → PATH 옵션 켜고 진행. 처음부터 다시 깔 필요 없음.

## ④ Claude Code — 이 수업의 주력 도구

①~③이 끝나야 설치 가능. 특히 Node.js가 먼저, 유료 Claude 구독도 필요.

(a) 설치 전 준비 — `node -v` 확인 / 유료 구독으로 [claude.com](https://claude.com) 로그인 확인

(b) 설치 — Day 1 「개발환경 구성」에서 PowerShell 네이티브 설치(`%USERPROFILE%\local\bin`)

💡 [확인 필요: 클로드 코드 공식 설치 명령 — 최신 형태를 [docs.claude.com](https://docs.claude.com)에서 확정. 네이티브 (PowerShell)·npm(`@anthropic-ai/claude-code`) 두 방식 가능]

- 설치 후 `claude --version` → 버전 나오면 성공 / 실행은 프로젝트 폴더에서 `claude`

## ④ Claude Code — 환경변수(PATH) 추가

`claude` 명령을 "찾을 수 없다"고 나올 때만.

1. 시작 → 환경 변수 검색 → "시스템 환경 변수 편집"
2. "시스템 속성" 창 아래 "환경 변수(N)..." 버튼
3. "사용자 변수"에서 "Path" → "편집(E)..." [캡처: Path 선택 상태]
4. "새로 만들기(N)" → `%USERPROFILE%\local\bin` 붙여넣기
5. 모든 창 "확인" → 터미널 완전히 껐다 새로 열고 `claude --version` 재시도

💡 혼자 씨름하지 말고 화면을 캡처해 강사에게 보이거나, Claude에게 물어보세요. 대부분 5분.

## 설치 가이드 — 이 장을 마치면

- [ ] `node -v` · `npm -v` 가 버전을 출력한다
- [ ] `git --version` 이 버전을 출력한다
- [ ] `python --version` 이 버전을 출력한다 ("Add Python to PATH" 체크함)
- [ ] `claude --version` 이 나오고, `claude` 로 클로드 코드가 실행된다

## AI가 만든 것 검증하기

### 부록

이론 03에서 못 박았듯 도구를 붙여도 AI는 틀릴 수 있습니다. 코드를 못 읽어도 확인할 수는 있습니다.

### 핵심 원칙 — 코드를 읽는 대신, 동작을 확인한다

- 인테리어 업체가 "끝났습니다!" 해도 수도·문·콘센트를 확인 — 도면 못 읽어도 검수는 가능
- 코드를 한 줄도 못 읽어도 동작을 확인하는 방법은 다섯 가지



## 방법 1 — 설명시키기

검증의 첫 수단은 코드가 아니라 **말입니다**. 설명이 시킨 것과 다르면 바로 수정 요청.

### EXAMPLE PROMPT

방금 만든 코드를 비개발자한테 설명하듯이 알려줘.

- 어떤 파일을 만들었거나 수정했는지
- 각각 무슨 역할인지
- 사용자 입장에서 뭐가 달라지는지

## 방법 2 — 이상한 입력을 넣어본다

- 일부러 틀리게 — 가격 칸에 "만원" 글자, 필수 항목 비우고 저장, 아주 긴 글 붙여넣기
- 문제를 찾으면 Day 1의 기대 vs 실제 형식으로 신고

### EXAMPLE PROMPT (문제 신고)

현재 상황: 가격에 "만원"이라고 글자를 입력했어.  
결과: 에러 없이 등록되는데 가격이 "NaN원"으로 표시돼.  
원하는 결과: 숫자만 입력 가능해야 하고, 글자를 넣으면 경고가 떠야 해.

## 방법 3 — 다른 AI에게 크로스체크

내가 판단할 필요 없이 AI들끼리 검증하게 만드는 방법.

순서	할 일
①	Claude에게 "결제 관련 코드 보여줘"
②	다른 AI(ChatGPT·Gemini)에 붙여넣고 "보안 문제·버그 있는지 확인"
③	지적받은 내용을 다시 Claude에게 — "이게 맞는지 확인하고, 문제 있으면 고쳐줘"

## 방법 4 — 보안 점검을 프롬프트 하나로

Day 2의 "키는 비밀번호다"를 이론이 아니라 행동으로.

### EXAMPLE PROMPT

이 프로젝트 보안 점검해줘:

1. API 키나 비밀번호가 코드에 직접 적혀 있진 않은지
  2. 브라우저에서 볼 수 있는 코드에 시크릿 키가 섞여 있진 않은지
  3. 깃허브에 올리면 안 되는 파일이 포함돼 있진 않은지
- 결과를 정리해서 알려줘.

# 방법 5 — 체크리스트를 통째로 AI에게

- 체크리스트는 사람이 읽는 것이라는 고정관념 버리기 — 항목을 적고 마지막에 한 줄: "이 체크리스트 대로 하나씩 확인해줘. 안 되는 게 있으면 고쳐줘."
- 교안의 「이 장을 마치면」 체크리스트들도 그대로 복사해 넘기면 됨

	내가 직접 써보기	AI에게 점검시키기
강점	"뭔가 이상한데" 직감, 쓰기 편한지	빠르고 빠짐없이 확인
약점	느리고 빠뜨림	사용성·느낌은 판단 못 함

 답은 둘 다 — AI에게 기본 점검, 내가 직접 써보며 느낌 확인.

## 검증 — 이 장을 마치면

- [ ] "코드를 못 읽는데 어떻게 확인하죠?"에 스스로 답할 수 있다
- [ ] 내 프로젝트에 이상한 입력을 넣어 최소 한 개의 문제를 찾아봤다
- [ ] 보안 점검 프롬프트를 한 번 돌려봤다

## 용어사전 — 쉬운 말 풀이

### 부록

수업 중 낯선 단어가 나오면 여기로. 목차 검색창에 단어를 쳐도 잡힙니다.

## 용어사전 — AI · 프롬프트 (1/2)

바이브코딩	말로 설명하면 AI가 만들어주는 개발 방식
프롬프트	AI에게 보내는 지시문. 구체적일수록 좋다
토큰	AI가 글을 읽고 쓰는 최소 단위. "다음 토큰"을 확률로 예측
컨텍스트 윈도우	한 번에 기억하는 대화의 양. 찰수록 흐려짐
환각	없는 사실을 그럴듯하게 지어내는 현상. 검증 필요
에이전트	다음 단계를 스스로 정해 도구를 쓰며 여러 걸음. 챗봇은 한 걸음



# 용어사전 — AI · 프롬프트 (2/2)

클로드 코드	터미널에서 도는 AI 코딩 에이전트. 파일을 읽고 고치고 실행
CLAUDE.md	프로젝트 규칙.맥락 파일. 매 대화마다 자동으로 읽힘
Skill(스킬)	절차를 등록해 /이름 으로 재사용. 엑셀 매크로
훅(Hook)	정해진 시점에 반드시 실행되는 자동 명령. 규칙이 부탁이면 훅은 강제
서브에이전트	다른 AI 작업자를 불러 나눠 시킴. 동시에 일해 빨라짐
MCP	외부 도구(브라우저·DB·문서)를 쫓는 표준 연결 규격

## 용어사전 — 웹 · 개발 (1/2)

프론트엔드	보이는 전부. HTML·CSS·JS
백엔드	화면 뒤 컴퓨터(서버). 로그인·주문 접수
데이터베이스(DB)	꺾다 켜도 남을 데이터 창고
API	다른 서비스의 데이터·기능을 쓰는 "주문 창구"
API 키	"요청할 자격이 있어요" 증명하는 열쇠
JSON	중괄호 { } 로 정리된 데이터 꾸러미 — API 응답 표준

# 용어사전 — 웹 · 개발 (2/2)

변수	값에 붙인 이름표 ( <code>const price = 3000</code> )
함수	이름 붙인 작업 묶음
조건문	"만약 ~라면"(if)
반복문	"각각에 대해"(for)
환경변수	컴퓨터 전체가 기억하는 설정 메모. Path·API 키 보관
터미널	명령어로 조작하는 창(PowerShell). 클로드 코드가 여기서 뚫

## 용어사전 — 배포 · 운영 (1/2)

깃(Git)	코드의 세이브 포인트. 망치면 되돌린다
깃허브(GitHub)	코드를 올려 백업·공유하는 클라우드
커밋 / 푸시	커밋 = 포장, 푸시 = 발송
GitHub Pages	저장소를 웹사이트로 켜주는 무료 기능
배포	인터넷 주소로 누구나 보게 공개
정적 페이지	서버·DB 없이 HTML만. GitHub Pages로
크롬 익스텐션	크롬에 설치하는 작은 프로그램

# 용어사전 — 배포 · 운영 (2/2)

PWA	홈 화면에 설치되고 전체화면으로 열리는 웹앱
토스페이먼츠	카드·간편결제를 붙여주는 한국 PG 서비스
테스트 결제	연습용 가짜 결제. 실제 돈 0원
MS Clarity	방문자 행동을 보여주는 무료 분석 도구
히트맵	많이 클릭된 곳일수록 빨강계. Clarity 대표 기능

# 용어사전 — 개발 환경·도구

IDE	코드편집기·터미널·확장·미리보기를 한 창에 모은 프로그램 (VS Code, 커서, 안티그래비티)
CLI	명령어를 글자로 입력해 다루는 방식. 클로드 코드가 이 방식
마크다운	텍스트만으로 문서를 쓰고 # · - 로 의미 부여. CLAUDE.md·README

# 용어사전 — TUI와 GUI

구분	GUI	TUI
풀이	그래픽 사용자 인터페이스	텍스트 기반 사용자 인터페이스
조작법	마우스로 클릭	키보드로 명령어 입력
예	탐색기, 크롬, 한글	터미널, 클로드 코드
강점	직관적, 배우기 쉬움	빠르고 자동화·기록에 강함
체감	이미 익숙	낯설지만 바이브코딩은 대부분 여기

## 용어사전 — 라이브러리·프레임워크·모듈·컴포넌트

용어	한마디	비유
라이브러리	필요한 기능을 꺼내 씀. 주도권은 나	도서관
프레임워크	정해진 뼈대 안에 내 코드를 끼움. 안정성↑	프랜차이즈
모듈	기능을 나눠 담은 파일 한 조각	레고 블록
컴포넌트	화면을 이루는 재사용 부품	조립 부품
API	다른 프로그램 기능을 부르는 창구	식당 주문 창구

💡 에이전트는 대부분 밑바닥이 아니라 검증된 라이브러리·프레임워크를 조립 — 이름을 알고 지시할수록 정확한 결과.



## 막혔을 때 응급처치

### 부록

시간을 잡아먹는 건 개발이 아니라 가입·설치·연결. 막히는 것은 여러분 잘못이 아닙니다.

### 무조건 먼저 해볼 3가지

- ① 에러 메시지(화면)를 그대로 복사/캡처 → "이 에러 해결해줘"
- ② 새 대화 열고 다시 시키기 — 길어지면 판단이 흐려짐
- ③ 프로그램(터미널·VS Code) 완전히 껐다 켜기 — "반영 안 됨"의 절반은 재시작

💡 막힘 3대 원인 — ① 계정 ② 연결 ③ 재시작 안 함. 이 순서로 점검.

# 응급처치 — 설치 · 명령어

증상	해결
claude 못 찾음	터미널 새로 열기 → PC 재시작 → Path에 %USERPROFILE%\local\bin
설치 명령이 안 먹힘	한 글자도 빠지 말고 그대로 복사했는지
로그인 브라우저 안 열림	터미널의 URL을 직접 복사해 주소창에
GitHub 가입 안 넘어감	스마트폰으로 가입 후 PC 로그인, 인증 코드는 복붙

# 응급처치 — GitHub Pages 배포

증상	해결
404 페이지	첫 화면 파일명이 정확히 <code>index.html</code> 인지 (대소문자까지)
고쳤는데 그대로	첫 배포 최대 10분. 뒤에 <b>Ctrl+F5</b>
Pages 메뉴 못 컴	Private이기 때문 — Settings → General → <b>Public</b>
모양이 깨짐	"CSS 파일 경로 확인해줘" (대소문자 불일치가 흔함)

# 응급처치 — API · 결제 / 공통

## API · 결제

증상	해결
응답 안 옴 / 키 에러	키 앞뒤 공백 확인 → 발급 사이트에서 승인 확인
결제 위젯 안 뜸	클라이언트 키 test_ck_ 시작·공백 확인
키 발급 자체가 안 됨	손 들기 — 공용 테스트 키

## 공통

증상	해결
하얀 빈 화면	F12 → Console 빨간 글씨 캡처해 Claude에게
3번 고쳤는데 계속 실패	"다른 방식으로" 또는 그 부분만 처음부터
아까는 됐는데 안 됨	완전 종료 후 재시작

💡 그래도 안 되면 손 들기 — 여러분이 막힌 곳은 다음 사람도 막힙니다.

# 하네스 엔지니어링

## AI를 더 잘 부리는 기술

### 부록

"하네스"는 말을 부릴 때 쓰는 장비. AI를 더 안전하고 정확하게 부리는 세팅과 습관.

### 핵심 개념 — "부탁"과 "강제"

	CLAUDE.md 규칙	훅(hook)
성격	부탁	강제
지켜지는 정도	거의 항상 (확률)	무조건 100% (자동 실행)
어울리는 일	말투, 스타일, 코딩 규칙	반드시 일어나야 하는 검사·차단

## AI를 잘 부리는 4가지 기술

기술	무엇을	예시
권한 설정	AI가 스스로 해도 되는 범위	"만들고 고치는 건 알아서, 지우는 건 물어보고"
계획 모드	실행 전 계획을 받아 검토	읽고 고친 뒤 실행
작업 쪼개기	큰 요청 1개 → 작은 여러 개	"쇼핑몰" 1번보다 "목록→장바구니→결제" 3번
서브에이전트	AI 작업자들을 동시에	30분 일이 10분에 (오늘은 개념만)

## 맡길까 말까 — 위임 판단 3질문

질문	예 → 맡겨도 좋다	아니오 → 사람이
되돌릴 수 있나?	코드 수정 (깃 복원)	메일 발송, 파일 삭제, 실제 결제
확인할 수 있나?	화면 열어 눈으로 검증	맞는지 확인할 방법 없음
규칙이 분명한가?	"이 형식으로 정리해줘"	정답 없는 가치 판단

💡 결제 예 — 연동 코드는 맡기고, 실제 돈이 오가는 최종 확인은 사람이. 최종 책임은 사람에게.

## 바이브코딩 핵심 습관 6가지

습관	왜
① 구체적으로 설명한다	비워둔 칸은 세상의 평균으로 채워짐
② 에러는 그대로 보여준다	에러 전문이 최고 단서
③ 작게 시작한다	큰 요청은 뒤로 갈수록 엉성
④ 기능 하나마다 실행해 확인	일찍 잡을수록 고치기 쉬움
⑤ "단계적으로 개발할 예정" 선언	완성 주문으로 오해 안 하게
⑥ 작업 하나 끝나면 새 대화	길어지면 판단이 흐려짐

⚠ 남의 혹 설정을 복사할 땐 반드시 내용 확인 — 혹은 자동 실행됨.



## 하네스 — 실습 & 이 장을 마치면

### 실습 — 바로 쓸 프로젝트 세팅

1. 새 폴더를 만들고 CLAUDE.md를 미리 작성 (어제 배운 것 활용)
2. 계획 모드로 "오늘 만들 것"의 계획을 받아 마음에 안 드는 부분 고치기

### 이 장을 마치면

- [ ] 프로젝트용 폴더와 CLAUDE.md가 준비돼 있다
- [ ] 계획 모드로 계획을 한 번 받아 고쳐봤다

참고 자료 — 훅(Hooks) 공식 가이드 / 서브에이전트 공식 문서

## 앱처럼 쓰는 반응형 웹

### 내 폰에 설치되는 웹앱

#### 부록

Day 1에 만든 프로필 페이지가 오늘 내 폰 홈 화면의 앱이 됩니다.

(도식: 설치되는 웹앱 흐름 도식)

## 설치되는 웹앱 — 핵심 내용

- 홈 화면에 설치되고 전체화면으로 열리는 웹 — 아이콘으로 실행하면 겉보기엔 일반 앱과 거의 같음
- 필요한 것은 둘 — manifest(앱 이름·아이콘) + 서비스워커(설치를 가능하게). 둘 다 Claude가 만듦
- 조건 하나 — https 주소에서만 설치. GitHub Pages는 자동 https라 어제 배포한 그대로면 이미 갖춰짐

## 앱스토어 앱과 뭐가 다른가

	설치되는 웹앱 (오늘)	앱스토어 앱
배포	심사·등록 없이 즉시	스토어 심사 (며칠~몇 주)
설치	"홈 화면에 추가"	스토어에서 다운로드
수정 반영	재배포하면 즉시	업데이트 심사 후
폰 기능	일부 제한 (알림 등)	전부 사용

- 오늘의 핵심은 "설치되는 경험"이므로 이 방식이 딱 맞다

## 실습 — 프로필 페이지를 앱으로

1. Day 1 프로필 폴더를 열고 아래 프롬프트 전송
2. 바뀐 파일들을 깃허브에 다시 올림 (Day 1 배포와 같은 방법)
3. 폰 브라우저로 내 주소 → 안드로이드(크롬): 메뉴(:) → 홈 화면에 추가 / 아이폰: 꼭 사파리로 → 공유 → 홈 화면에 추가
4. 홈 화면 아이콘을 눌러 전체화면으로 열리면 성공

### EXAMPLE PROMPT

이 프로필 페이지를 홈 화면에 설치되는 웹앱으로 만들어줘.  
홈 화면에 설치할 수 있게 `manifest`와 서비스워커를 추가하고,  
앱 이름은 "OO의 프로필", 아이콘도 간단하게 만들어줘.

## 웹앱 — 이 장을 마치면

- [ ] 내 폰 홈 화면에 내가 만든 앱 아이콘이 있다
- [ ] 아이콘을 누르면 주소창 없이 전체화면으로 열린다

**참고 자료** — 홈 화면에 설치되는 웹앱 만들기 (MDN, 한국어)

## MS Clarity

### 실사용자는 내 서비스를 어떻게 쓰는가

#### 부록

"만들었다"에서 끝나지 않고, 사람들이 실제로 어떻게 쓰는지 보고 고치는 것 — 그게 서비스 운영입니다.

(도식: Clarity 사용자 추적 흐름 도식)

# MS Clarity — 핵심 내용

- MS Clarity = 마이크로소프트가 무료 제공하는 사용자 행동 분석 도구. 어디를 누르고.어디까지 읽고. 어디서 나가는지
- 오늘 시연은 미리 설치해 데이터가 쌓인 사이트로 — 설치 후 대시보드에 뜨기까지 수십 분~수 시간 (내 데이터는 내일 확인)

기능	보여주는 것	보고 나서 할 일
히트맵	많이 클릭된 곳일수록 빨강계	아무도 안 누르는 버튼 없앴
세션 녹화	방문자 쓰는 모습을 영상처럼	이탈하는 지점 고침
분노의 클릭	안 되는 버튼 연타 흔적	고장.헛갈리는 버튼 고침



## 실습 — 내 사이트에 설치

1. clarity.microsoft.com → 마이크로소프트/구글 계정 가입
2. 새 프로젝트 → 이름·배포 주소 입력 → 설정 → 설치 → "직접 설치" 탭의 추적 코드 통째로 복사
3. Claude에게 "이 스크립트를 내 페이지의 head에 넣어줘" + 코드 붙여넣기
4. 깃허브에 다시 올려 재배포 — 데이터는 오늘 저녁~내일 대시보드에서

### 이 장을 마치면

- [ ] 추적 코드가 들어간 내 사이트가 다시 배포돼 있다
- [ ] 내일 대시보드에서 무엇을 볼지(히트맵·세션 녹화) 안다

참고 자료 — MS Clarity (가입·대시보드) / Clarity 공식 문서